คู่มือเล่ม 1: ปฐมบทสถาปัตยกรรมซอฟต์แวร์

(Software Architecture Foundations)

---

1. ข้อมูลเล่ม

รายการ	รายละเอียด
ชื่อเล่ม	ปฐมบทสถาปัตยกรรมซอฟต์แวร์
ชื่อรอง	พื้นฐาน ภาษากลาง และการเชื่อมสถาปัตยกรรมสู่โค้ด
กลุ่มผู้อ่าน	Software Architect, Tech Lead, Senior Developer, Tester, DevOps, PM ที่ต้องทำงานกับสถาปัตยกรรม
ระดับ	พื้นฐานถึงกลาง
ความสัมพันธ์กับเล่มอื่น	เป็นเล่มเปิดชุด ใช้อ้างอิงร่วมกับเล่ม 2–7
โครงสร้างหลักที่ใช้	เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป

---

2. โครงสร้างระดับเล่ม

2.1 เรื่อง

เล่มนี้ว่าด้วย “ภาษากลาง” ของสถาปัตยกรรมซอฟต์แวร์ ตั้งแต่แนวคิดพื้นฐาน รูปแบบสถาปัตยกรรม (architectural styles/patterns) การจัดองค์ประกอบ การ dokument สถาปัตยกรรม และการเชื่อมสถาปัตยกรรมลงสู่โค้ดจริง โดยเน้นให้ผู้อ่านแยกแยะได้ว่า อะไรคือสถาปัตยกรรม อะไรคือ pattern อะไรคือ design pattern และจะทำให้สถาปัตยกรรมปรากฏในโค้ดอย่างตรวจสอบได้อย่างไร

2.2 วัตถุประสงค์

1. อธิบายคำศัพท์และแนวคิดพื้นฐานของสถาปัตยกรรมซอฟต์แวร์
2. แยกความแตกต่างระหว่าง architectural style, architectural pattern และ design pattern
3. เข้าใจสถาปัตยกรรมแบบ monolithic, component-based, layered และรูปแบบที่เกี่ยวข้อง

4. อ่านและเขียน diagram สถาปัตยกรรมอย่างง่ายได้
5. เชื่อมสถาปัตยกรรมกับโครงสร้างโค้ด การแบ่ง package/component และการทดสอบ
6. วางรากฐานก่อนเข้าสู่ Microservices, DDD และ Clean Architecture ในเล่มถัดไป

2.3 เป้าหมาย

เมื่อจบเล่ม ผู้อ่านสามารถ:

- อธิบายสถาปัตยกรรมของระบบหนึ่งด้วยภาษากลางได้
- เลือกใช้ style/pattern เบื้องต้นพร้อมเหตุผลและข้อจำกัด
- วาด diagram สถาปัตยกรรมระดับ context/component ได้
- ตรวจสอบว่าโค้ดสะท้อนสถาปัตยกรรมหรือไม่
- ตั้งคำถามและเกณฑ์ตัดสินใจเชิงสถาปัตยกรรมได้
- ส่งต่องานให้ทีม DDD, Microservices และ Clean Architecture ได้อย่างสอดคล้อง

2.4 ข้อดีของเล่ม

- เป็นเล่มฐานที่ทำให้เล่มอื่นอ่านง่ายขึ้น
- รวมเนื้อหาที่มักกระจัดกระจายไว้ในที่เดียว
- มีตัวอย่าง diagram, package structure และ case study
- ใช้ได้ทั้งทีมพัฒนา ทีมทดสอบ และผู้บริหารโครงการ
- มีแม่แบบและ checklist ที่นำไปใช้ได้ทันที

2.5 ข้อเสีย/ข้อจำกัด

- ไม่ได้ลงลึกถึงระดับ implementation ทุกเทคโนโลยี
- Microservices, DDD และ Clean Architecture มีเพียงบทเชื่อม ไม่ใช่เนื้อหาหลัก
- ตัวอย่างอาจต้องปรับตามบริบทองค์กร
- ผู้อ่านที่ไม่มีประสบการณ์โค้ดจริงอาจต้องอ่านตัวอย่างเพิ่ม

2.6 ข้อควรระวัง

- อย่าใช้ pattern เพราะ “นิยม” โดยไม่ดู architectural drivers
- อย่าสับสนระหว่าง diagram กับสถาปัตยกรรมจริง
- อย่าปล่อยให้สถาปัตยกรรมกับโค้ดไม่ตรงกัน
- ระวังการแบ่ง layer/package ตามเทคโนโลยีจนธุรกิจถูกซ่อน
- ต้องอัปเดตเอกสารเมื่อสถาปัตยกรรมเปลี่ยน

2.7 ตัวอย่างที่ใช้ในเล่ม

- ระบบ monolith ขนาดเล็ก
- ระบบ component-based

- การแบ่ง package ตาม component vs ตาม layer
- Diagram แบบ simple sketch
- Case study: techtribes.je – context
- ตัวอย่างการ mapping architecture → code

2.8 การประยุกต์ใช้

- ใช้เป็นคู่มือประชุม architecture review
- ใช้ตรวจสอบโครงสร้าง package ก่อนเริ่ม refactor
- ใช้เป็นสื่อกลางระหว่าง architect, dev และ tester
- ใช้ตั้ง architectural drivers และ decision record
- ใช้เป็นพื้นฐานก่อนทำ POC หรือเลือก Microservices

2.9 สรุป

เล่ม 1 ทำหน้าที่เป็น “แผนที่นำทาง” ของทั้งชุด ผู้อ่านจะได้ทั้งคำศัพท์ ภาพรวมรูปแบบสถาปัตยกรรม วิธี dokument และวิธีทำให้สถาปัตยกรรมสะท้อนในโค้ด เพื่อให้การตัดสินใจในเล่ม 2–7 มีพื้นฐานร่วมกัน

---

3. สารบัญละเอียด

ส่วนที่ 1: ภาพรวมและภาษากลางของสถาปัตยกรรม

บทที่ 1 The Software Architecture Chronicles

- เรื่อง: สถาปัตยกรรมซอฟต์แวร์คืออะไร และทำไมต้องสนใจ
- วัตถุประสงค์: สร้างภาพรวมและคำนิยามร่วม
- หัวข้อ:
  - ความหมายของ software architecture
  - บทบาทของ architect
  - สถาปัตยกรรมกับความต้องการทางธุรกิจ
  - วงจรชีวิตของสถาปัตยกรรม
- ตัวอย่าง: ระบบหนึ่งที่มีการเปลี่ยนแปลงจาก monolith → component-based
- ข้อควรระวัง: อย่านิยามสถาปัตยกรรมแค่เรื่องเทคโนโลยี
- ผลลัพธ์: ผู้อ่านมีภาษากลางเบื้องต้น

บทที่ 2 Software Architecture Premises

- เรื่อง: ข้อตั้งต้นและสมมติฐานของสถาปัตยกรรม
- หัวข้อ:

    - Quality attributes
    - Constraints
    - Architectural drivers
    - Trade-offs
- **ตัวอย่าง:** การเลือก scalability vs simplicity
- **ข้อควรระวัง:** สมมติฐานที่ไม่ถูกตรวจสอบจะกลายเป็นความเสี่ยง
- **ผลลัพธ์:** ระบุ drivers ของระบบได้

บทที่ 3 Architectural Styles vs. Architectural Patterns vs. Design Patterns

- **เรื่อง:** แยกประเภทคำที่มักสับสน
- **หัวข้อ:**
    - Style คืออะไร
    - Pattern คืออะไร
    - Design pattern ต่างกันอย่างไร
    - ตัวอย่างการจัดหมวด
- **ตัวอย่าง:** Layered style, MVC pattern, Strategy design pattern
- **ข้อควรระวัง:** ใช้คำผิดระดับทำให้การตัดสินใจคลาดเคลื่อน
- **ผลลัพธ์:** จัดประเภทแนวทางสถาปัตยกรรมได้ถูกต้อง

บทที่ 4 Architectural Patterns and Styles

- **เรื่อง:** ภาพรวมรูปแบบสถาปัตยกรรมที่พบบ่อย
- **หัวข้อ:**
    - Layered
    - Component-based
    - Monolithic
    - Event-driven เบื้องต้น
    - Microservices เบื้องต้น
    - Hexagonal/Onion/Clean เบื้องต้น
- **ตัวอย่าง:** เปรียบเทียบ style ในระบบเดียวกัน
- **ข้อควรระวัง:** แต่ละ style มีบริบทเหมาะสมต่างกัน
- **ผลลัพธ์:** เลือก style เบื้องต้นพร้อมเหตุผลได้

บทที่ 5 Architectural Drivers: Building Blocks for Decision Making

- **เรื่อง:** ตัวขับเคลื่อนการตัดสินใจทางสถาปัตยกรรม
- **หัวข้อ:**
    - Functional requirements vs quality attributes

    - Business drivers
    - Technical drivers
    - การจัดลำดับความสำคัญ
- ตัวอย่าง: ระบบที่ต้องรองรับผู้ใช้จำนวนมากแต่ทีมเล็ก
- ข้อควรระวัง: ทุกอย่างเป็น priority สูงไม่ได้
- ผลลัพธ์: ทำ Architecture Driver List ได้

ส่วนที่ 2: รูปแบบสถาปัตยกรรมและการจัดองค์ประกอบ

บทที่ 6 Monolithic Architecture

- เรื่อง: สถาปัตยกรรมแบบ monolith
- หัวข้อ:
    - โครงสร้างโดยทั่วไป
    - ข้อดี: ง่ายต่อการเริ่มต้น, deploy ง่าย
    - ข้อเสีย: coupling, scaling แบบทั้งก้อน
    - เมื่อใดควรใช้
- ตัวอย่าง: ระบบ ERP ขนาดเล็ก
- ข้อควรระวัง: monolith ไม่ใช่สิ่งเลวร้ายเสมอ
- ผลลัพธ์: ประเมินความเหมาะสมของ monolith ได้

บทที่ 7 Component Based Architecture

- เรื่อง: การออกแบบด้วย component
- หัวข้อ:
    - Component คืออะไร
    - Boundary และ contract
    - Components vs classes
    - การสื่อสารระหว่าง component
- ตัวอย่าง: ระบบ e-commerce แยก catalog, order, payment
- ข้อควรระวัง: component ที่ใหญ่เกินไปหรือเล็กเกินไป
- ผลลัพธ์: วาด component diagram เบื้องต้นได้

บทที่ 8 PresentationDomainDataLayering

- เรื่อง: การแบ่งชั้นแบบ Presentation–Domain–Data
- หัวข้อ:
    - แต่ละชั้นรับผิดชอบอะไร

    - Dependency ระหว่างชั้น
    - ข้อดี/ข้อเสีย
- ตัวอย่าง: Web app ที่แยก UI, business logic, data access
- ข้อควรระวัง: domain กลายเป็นชั้นบาง ๆ ที่ไม่มี logic
- ผลลัพธ์: ออกแบบ layer เบื้องต้นได้

บทที่ 9 Layers, Hexagons, Features and Components

- เรื่อง: ทางเลือกในการจัดโครงสร้าง นอกจาก layer
- หัวข้อ:
    - Layer-based
    - Hexagonal เบื้องต้น
    - Feature-based
    - Component-based
    - เปรียบเทียบข้อดี/ข้อเสีย
- ตัวอย่าง: โครงสร้าง package แบบต่าง ๆ
- ข้อควรระวัง: การจัดตาม layer อาจซ่อน domain
- ผลลัพธ์: เลือกแนวทางจัดโครงสร้างได้

บทที่ 10 More than Concentric Layers

- เรื่อง: เชื่อมจาก layer สู่แนวคิดโดเมน
- หัวข้อ:
    - ข้อจำกัดของ concentric layers
    - ทำไมต้องสะท้อนโดเมน
    - บทนำไปสู่ DDD/Clean Architecture
- ตัวอย่าง: ระบบที่มี business rules ซับซ้อน
- ข้อควรระวัง: อย่าแยกชั้นจน business logic กระจัดกระจาย
- ผลลัพธ์: เข้าใจสะพานเชื่อมไปเล่ม 3–4

ส่วนที่ 3: จากสถาปัตยกรรมสู่โค้ด

บทที่ 11 Software Architecture vs Code

- เรื่อง: ความสัมพันธ์ระหว่างสถาปัตยกรรมกับโค้ด
- หัวข้อ:
    - สถาปัตยกรรมปรากฏในโค้ดอย่างไร
    - drift ระหว่าง design กับ code

    - วิธีตรวจสอบ
- ตัวอย่าง: package ที่ไม่ตรงกับ component diagram
- ข้อควรระวัง: เอกสารกับโค้ดไม่ตรงกัน
- ผลลัพธ์: ตรวจ alignment ได้

บทที่ 12 Mapping Software Architecture to Code

- เรื่อง: การแปลงสถาปัตยกรรมเป็นโครงสร้างโค้ด
- หัวข้อ:
    - Component → package/module
    - Interface/contract
    - Dependency direction
    - การตั้งชื่อ
- ตัวอย่าง: mapping component diagram สู่ package
- ข้อควรระวัง: mapping แบบ 1:1 อาจไม่เหมาะทุกกรณี
- ผลลัพธ์: ทำ architecture-to-code map ได้

บทที่ 13 An Architecturally-Evident Coding Style

- เรื่อง: สไตล์โค้ดที่ทำให้เห็นสถาปัตยกรรม
- หัวข้อ:
    - ชื่อ package สื่อความหมาย
    - การจัดกลุ่มไฟล์
    - dependency ที่มองเห็นได้
    - สัญญาณเตือนเมื่อโค้ดเบี่ยงจากสถาปัตยกรรม
- ตัวอย่าง: โค้ดที่อ่านแล้วเห็น layer/component
- ข้อควรระวัง: ชื่อสวยแต่โครงสร้างไม่จริง
- ผลลัพธ์: ตั้ง coding style ที่สะท้อนสถาปัตยกรรมได้

บทที่ 14 Components vs Classes

- เรื่อง: ระดับการออกแบบที่ต่างกัน
- หัวข้อ:
    - Class คือหน่วยโค้ด
    - Component คือหน่วยสถาปัตยกรรม
    - ความสัมพันธ์และการแบ่งหน้าที่
- ตัวอย่าง: class หลายตัวรวมเป็น component เดียว
- ข้อควรระวัง: ใช้ class แทน component จน boundary หาย
- ผลลัพธ์: แยกหน่วยโค้ดกับหน่วยสถาปัตยกรรมได้

บทที่ 15 Reflecting Architecture and Domain in Code

- เรื่อง: ให้โค้ดสะท้อนทั้งสถาปัตยกรรมและโดเมน
- หัวข้อ:
  - โครงสร้างที่สื่อ domain
  - โครงสร้างที่สื่อ architecture
  - การหลีกเลี่ยงโครงสร้างตาม framework
- ตัวอย่าง: package ตาม domain + component
- ข้อควรระวัง: พยายามสะท้อนทุกอย่างจนซับซ้อนเกิน
- ผลลัพธ์: ออกแบบโครงสร้างโค้ดที่สื่อความหมายได้

ส่วนที่ 4: Modularity, Packaging และการทดสอบ

บทที่ 16 Modularity and Testability

- เรื่อง: ความเชื่อมโยงระหว่าง modularity กับการทดสอบ
- หัวข้อ:
  - Coupling/cohesion
  - Test pyramid เบื้องต้น
  - การทดสอบ component
  - Dependency injection เบื้องต้น
- ตัวอย่าง: component ที่ทดสอบง่าย vs ยาก
- ข้อควรระวัง: modularity ที่ไม่จริงทำให้ทดสอบยาก
- ผลลัพธ์: ประเมิน testability ได้

บทที่ 17 Package by Component and Architecturally-Aligned Testing

- เรื่อง: จัด package ตาม component และทดสอบให้สอดคล้อง
- หัวข้อ:
  - Package by component
  - Testing ตาม boundary
  - Contract test เบื้องต้น
- ตัวอย่าง: package structure และชุดทดสอบ
- ข้อควรระวัง: ทดสอบข้าม boundary โดยไม่ตั้งใจ
- ผลลัพธ์: ออกแบบ package และ test strategy ได้

บทที่ 18 Schools of Package Architecture – An Illustration

- เรื่อง: แนวทางจัด package หลายสำนัก

- หัวข้อ:
    - Package by layer
    - Package by feature
    - Package by component
    - Package by domain
    - ข้อดี/ข้อเสียแต่ละแบบ
- ตัวอย่าง: ระบบเดียวกันจัด package 4 แบบ
- ข้อควรระวัง: ไม่มีแบบเดียวที่เหมาะทุกทีม
- ผลลัพธ์: เลือกแนวทาง package ได้อย่างมีเหตุผล

บทที่ 19 Aligning Software Architecture and Code

- เรื่อง: ทำให้สถาปัตยกรรมและโค้ดสอดคล้องกันอย่างต่อเนื่อง
- หัวข้อ:
    - Architecture review
    - Fitness function เบื้องต้น
    - Static analysis
    - Refactoring เพื่อรักษา alignment
- ตัวอย่าง: ตรวจ dependency rule ด้วยเครื่องมือ
- ข้อควรระวัง: ปล่อย drift สะสมจนแก้ยาก
- ผลลัพธ์: วางกลไกกำกับ alignment ได้

ส่วนที่ 5: การ dokument และกรณีศึกษา

บทที่ 20 Documenting Software Architecture

- เรื่อง: การ dokument สถาปัตยกรรมอย่างพอดี
- หัวข้อ:
    - Document อะไร
    - สำหรับใคร
    - Context diagram, component diagram, deployment diagram
    - Architecture Decision Record (ADR) เบื้องต้น
- ตัวอย่าง: ชุดเอกสารสถาปัตยกรรมขนาดเล็ก
- ข้อควรระวัง: เอกสารมากเกินไปแต่ไม่ถูกใช้
- ผลลัพธ์: จัดทำเอกสารสถาปัตยกรรมขั้นต่ำได้

บทที่ 21 Simple Sketches for Diagramming your Software Architecture

- เรื่อง: การวาด diagram อย่างง่ายแต่สื่อสารได้
- หัวข้อ:
  - สัญลักษณ์พื้นฐาน
  - Context → Container → Component
  - การแสดง dependency
  - การหลีกเลี่ยง diagram ที่รก
- ตัวอย่าง: simple sketch ของระบบหนึ่ง
- ข้อควรระวัง: diagram ไม่มี legend หรือขอบเขต
- ผลลัพธ์: วาด diagram เพื่อสื่อสารได้

บทที่ 22 Case Study: techtribes.je – Context

- เรื่อง: กรณีศึกษาการทำความเข้าใจ context
- หัวข้อ:
  - System context
  - ผู้ใช้และระบบภายนอก
  - Constraints
  - Architectural drivers
- ตัวอย่าง: techtribes.je
- ข้อควรระวัง: มองข้ามระบบภายนอกและผู้ใช้งาน
- ผลลัพธ์: เขียน context diagram และ driver list ได้

บทที่ 23 การประยุกต์ใช้และสรุปเล่ม

- เรื่อง: นำทั้งหมดไปใช้
- หัวข้อ:
  - Workshop: Architecture Context Canvas
  - Workshop: Style/Pattern Decision
  - Workshop: Package Alignment Review
  - สรุปสาระสำคัญ
  - เชื่อมไปเล่ม 2–4
- ตัวอย่าง: แผนนำไปใช้ใน 30 วัน
- ข้อควรระวัง: อย่านำไปใช้แบบคัดลอกโดยไม่ปรับบริบท
- ผลลัพธ์: แผนประยุกต์ใช้ในองค์กร

---

4. แม่แบบและเครื่องมือท้ายเล่ม

1. **Architecture Context Canvas**
    - ระบบคืออะไร
    - ผู้ใช้/actor
    - ระบบภายนอก
    - Constraints
    - Quality attributes
    - Architectural drivers
2. **Style/Pattern Decision Record**
    - ปัญหา
    - ทางเลือก
    - เหตุผล
    - ข้อดี/ข้อเสีย
    - ความเสี่ยง
    - การตัดสินใจ
3. **Component & Dependency Map Template**
    - Component
    - ความรับผิดชอบ
    - Interface/contract
    - Dependency
    - Owner
4. **Package Alignment Checklist**
    - Package สื่อ component/domain หรือไม่
    - Dependency direction ถูกต้องหรือไม่
    - มี cyclic dependency หรือไม่
    - Test ตรง boundary หรือไม่
5. **Diagram Checklist**
    - มีชื่อ diagram
    - มี legend
    - มีขอบเขต
    - มีทิศทาง dependency
    - มีคำอธิบายประกอบ
6. **Glossary กลาง**

- Architecture, Style, Pattern, Design Pattern, Component, Module, Package, Layer, Boundary, Driver, Quality Attribute, ADR ฯลฯ

---

5. เกณฑ์ประเมินความสำเร็จของเล่ม

เกณฑ์	ตัวชี้วัด
ความเข้าใจ	ผู้อ่านอธิบาย style/pattern/design pattern ต่างกันได้
การนำไปใช้	มี diagram และ package map ของระบบตนเอง
การตัดสินใจ	มี decision record อย่างน้อย 1 ฉบับ
ความสอดคล้อง	ตรวจ alignment ระหว่างสถาปัตยกรรมกับโค้ดได้
การส่งต่อ	เข้าใจว่าบทใดเชื่อมไปเล่ม 2, 3, 4

---

6. การเชื่อมโยงกับเล่มอื่น

หัวข้อในเล่ม 1	เชื่อมไปเล่ม
Microservices เบื้องต้น	เล่ม 2
DDD และ domain ในโค้ด	เล่ม 3
Clean Architecture / concentric layers	เล่ม 4
ความเสี่ยงจากการตัดสินใจสถาปัตยกรรม	เล่ม 5
การทำ POC เลือก style/pattern	เล่ม 6
การบริหารงานสถาปัตยกรรมในโครงการ	เล่ม 7

---

7. สรุปเล่ม 1

เล่ม 1 เป็นฐานของทั้งชุด: ทำให้ผู้อ่านมีภาษากลาง เข้าใจรูปแบบสถาปัตยกรรม เห็นความเชื่อมโยงระหว่างสถาปัตยกรรมกับโค้ด และมีเครื่องมือพื้นฐานสำหรับ dokument และตรวจสอบความสอดคล้อง ก่อนจะลงลึกใน Microservices, DDD, Clean Architecture, ความเสี่ยง, POC และการบริหารโครงการในเล่มถัดไป

ด้านล่างคือ **สารบัญเล่ม 1 ในรูปแบบไฟล์ Markdown พร้อมโครงแต่ละบทแบบละเอียด**
ชื่อไฟล์แนะนำ: `LE1-Software-Architecture-Foundations.md`

```
# คู่มือเล่ม 1: ปฐมบทสถาปัตยกรรมซอฟต์แวร์
## Software Architecture Foundations

> ชุดคู่มือสถาปัตยกรรมซอฟต์แวร์และการบริหารโครงการ
> เล่ม 1 จาก 7 เล่ม
> สถานะ: Draft v0.1

---

## ข้อมูลเอกสาร

| รายการ | รายละเอียด |
|---|---|
| ชื่อเล่ม | ปฐมบทสถาปัตยกรรมซอฟต์แวร์ |
| ชื่อรอง | พื้นฐาน ภาษากลาง และการเชื่อมสถาปัตยกรรมสู่โค้ด |
| กลุ่มผู้อ่าน | Software Architect, Tech Lead, Senior Developer, Tester, DevOps, PM |
| ระดับ | พื้นฐานถึงกลาง |
| โครงสร้างหลัก | เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป |
| เล่มที่เชื่อมโยง | เล่ม 2 Microservices, เล่ม 3 DDD, เล่ม 4 Clean Architecture, เล่ม 5 ความเสี่ยง, เล่ม 6 POC, เล่

---

## วิธีใช้เอกสาร

1. ใช้บทที่ 1–5 เพื่อสร้างภาษากลางและความเข้าใจพื้นฐาน
2. ใช้บทที่ 6–10 เพื่อเลือกและเปรียบเทียบรูปแบบสถาปัตยกรรม
3. ใช้บทที่ 11–15 เพื่อเชื่อมสถาปัตยกรรมลงสู่โค้ด
4. ใช้บทที่ 16–19 เพื่อตรวจสอบ modularity, package และการทดสอบ
5. ใช้บทที่ 20–23 เพื่อ dokument, วาด diagram และนำไปประยุกต์ใช้
6. ใช้แม่แบบท้ายเล่มเป็นเครื่องมือประกอบ workshop และ architecture review

---

## โครงสร้างบทมาตรฐาน

ทุกบทในเล่มนี้ใช้โครงสร้างร่วมกันดังนี้

1. เรื่อง
2. วัตถุประสงค์
3. เป้าหมาย
4. หัวข้อหลัก
5. ข้อดีของบท
```

6. ข้อเสีย/ข้อจำกัด
7. ข้อควรระวัง
8. ตัวอย่าง
9. การประยุกต์ใช้
10. สรุป/ผลลัพธ์
11. เชื่อมโยงกับเล่มอื่น

สารบัญ

ส่วนที่ 1: ภาพรวมและภาษากลางของสถาปัตยกรรม
- บทที่ 1 The Software Architecture Chronicles
- บทที่ 2 Software Architecture Premises
- บทที่ 3 Architectural Styles vs. Architectural Patterns vs. Design Patterns
- บทที่ 4 Architectural Patterns and Styles
- บทที่ 5 Architectural Drivers: Building Blocks for Decision Making

ส่วนที่ 2: รูปแบบสถาปัตยกรรมและการจัดองค์ประกอบ
- บทที่ 6 Monolithic Architecture
- บทที่ 7 Component Based Architecture
- บทที่ 8 PresentationDomainDataLayering
- บทที่ 9 Layers, Hexagons, Features and Components
- บทที่ 10 More than Concentric Layers

ส่วนที่ 3: จากสถาปัตยกรรมสู่โค้ด
- บทที่ 11 Software Architecture vs Code
- บทที่ 12 Mapping Software Architecture to Code
- บทที่ 13 An Architecturally-Evident Coding Style
- บทที่ 14 Components vs Classes
- บทที่ 15 Reflecting Architecture and Domain in Code

ส่วนที่ 4: Modularity, Packaging และการทดสอบ
- บทที่ 16 Modularity and Testability
- บทที่ 17 Package by Component and Architecturally-Aligned Testing
- บทที่ 18 Schools of Package Architecture – An Illustration
- บทที่ 19 Aligning Software Architecture and Code

ส่วนที่ 5: การ dokument และกรณีศึกษา
- บทที่ 20 Documenting Software Architecture
- บทที่ 21 Simple Sketches for Diagramming your Software Architecture
- บทที่ 22 Case Study: techtribes.je – Context
- บทที่ 23 การประยุกต์ใช้และสรุปเล่ม

ภาคผนวก
- ภาคผนวก A: แม่แบบ Architecture Context Canvas
- ภาคผนวก B: แม่แบบ Style/Pattern Decision Record

- ภาคผนวก C: แม่แบบ Component & Dependency Map
- ภาคผนวก D: Package Alignment Checklist
- ภาคผนวก E: Diagram Checklist
- ภาคผนวก F: Glossary กลาง

สารบัญละเอียดพร้อมโครงแต่ละบท

ส่วนที่ 1: ภาพรวมและภาษากลางของสถาปัตยกรรม

บทที่ 1 The Software Architecture Chronicles

- **เรื่อง:** สถาปัตยกรรมซอฟต์แวร์คืออะไร และทำไมต้องให้ความสำคัญ
- **วัตถุประสงค์:** สร้างคำนิยามร่วมและภาพรวมของสถาปัตยกรรมซอฟต์แวร์
- **เป้าหมาย:** ผู้อ่านอธิบายความหมาย บทบาท และคุณค่าของสถาปัตยกรรมได้
- **หัวข้อหลัก:**
  - ความหมายของ software architecture
  - บทบาทของ architect
  - สถาปัตยกรรมกับความต้องการทางธุรกิจ
  - วงจรชีวิตของสถาปัตยกรรม
  - สถาปัตยกรรมกับโค้ดและทีม
- **ข้อดีของบท:** เป็นจุดเริ่มต้นที่สร้างภาษากลางให้ทั้งเล่ม
- **ข้อเสีย/ข้อจำกัด:** ยังไม่ลงรายละเอียดเชิงเทคนิค
- **ข้อควรระวัง:** อย่านิยามสถาปัตยกรรมเพียงเรื่องเทคโนโลยีหรือ diagram
- **ตัวอย่าง:** ระบบหนึ่งที่มีการเปลี่ยนแปลงจาก monolith สู่ component-based
- **การประยุกต์ใช้:** ใช้เปิดประชุม architecture review และตั้งคำนิยามร่วมของทีม
- **สรุป/ผลลัพธ์:** ผู้อ่านมีภาษากลางเบื้องต้นเกี่ยวกับสถาปัตยกรรม
- **เชื่อมโยง:** ไปสู่บทที่ 2–4 และเล่ม 2–4

บทที่ 2 Software Architecture Premises

- **เรื่อง:** ข้อตั้งต้น สมมติฐาน และข้อจำกัดของสถาปัตยกรรม
- **วัตถุประสงค์:** ทำให้ผู้อ่านตระหนักถึงปัจจัยที่อยู่เบื้องหลังการตัดสินใจ
- **เป้าหมาย:** ระบุ quality attributes, constraints และ trade-offs ของระบบได้
- **หัวข้อหลัก:**
  - Quality attributes
  - Constraints
  - Architectural drivers
  - Trade-offs
  - สมมติฐานที่ต้องตรวจสอบ
- **ข้อดีของบท:** ทำให้การตัดสินใจสถาปัตยกรรมมีเหตุผลรองรับ
- **ข้อเสีย/ข้อจำกัด:** ต้องอาศัยข้อมูลธุรกิจและเทคนิคประกอบ
- **ข้อควรระวัง:** สมมติฐานที่ไม่ถูกตรวจสอบจะกลายเป็นความเสี่ยง
- **ตัวอย่าง:** การเลือก scalability vs simplicity ในระบบเดียวกัน

- **การประยุกต์ใช้:** จัดทำรายการสมมติฐานและข้อจำกัดก่อนออกแบบ
- **สรุป/ผลลัพธ์:** ผู้อ่านระบุ drivers และ premises ของระบบได้
- **เชื่อมโยง:** ไปสู่บทที่ 5 และเล่ม 5–6

บทที่ 3 Architectural Styles vs. Architectural Patterns vs. Design Patterns

- **เรื่อง:** การแยกความแตกต่างของคำที่มักใช้สับสน
- **วัตถุประสงค์:** สร้างความชัดเจนเรื่องระดับของ style, pattern และ design pattern
- **เป้าหมาย:** จัดประเภทแนวทางสถาปัตยกรรมได้ถูกต้อง
- **หัวข้อหลัก:**
  - Architectural style คืออะไร
  - Architectural pattern คืออะไร
  - Design pattern คืออะไร
  - ระดับการนำไปใช้
  - ตัวอย่างการจัดหมวด
- **ข้อดีของบท:** ลดความสับสนในทีมและเอกสาร
- **ข้อเสีย/ข้อจำกัด:** คำนิยามในอุตสาหกรรมอาจแตกต่างกันบ้าง
- **ข้อควรระวัง:** ใช้คำผิดระดับทำให้การตัดสินใจคลาดเคลื่อน
- **ตัวอย่าง:** Layered style, MVC pattern, Strategy design pattern
- **การประยุกต์ใช้:** ใช้ตรวจทานเอกสารสถาปัตยกรรมและ glossary ของทีม
- **สรุป/ผลลัพธ์:** ผู้อ่านแยกประเภทแนวทางสถาปัตยกรรมได้
- **เชื่อมโยง:** ไปสู่บทที่ 4 และเล่ม 4

บทที่ 4 Architectural Patterns and Styles

- **เรื่อง:** ภาพรวมรูปแบบสถาปัตยกรรมที่พบบ่อย
- **วัตถุประสงค์:** แนะนำ style/pattern หลักและบริบทการใช้งาน
- **เป้าหมาย:** เลือก style/pattern เบื้องต้นพร้อมเหตุผลได้
- **หัวข้อหลัก:**
  - Layered
  - Component-based
  - Monolithic
  - Event-driven เบื้องต้น
  - Microservices เบื้องต้น
  - Hexagonal/Onion/Clean เบื้องต้น
  - การเปรียบเทียบข้อดี/ข้อเสีย
- **ข้อดีของบท:** เป็นแผนที่นำทางก่อนลงลึกในเล่มถัดไป
- **ข้อเสีย/ข้อจำกัด:** แต่ละรูปแบบมีรายละเอียดมาก จึงเป็นเพียงภาพรวม
- **ข้อควรระวัง:** อย่าเลือก pattern ตามกระแสโดยไม่ดู drivers
- **ตัวอย่าง:** เปรียบเทียบ style หลายแบบในระบบเดียวกัน
- **การประยุกต์ใช้:** ใช้ใน workshop เลือกแนวทางสถาปัตยกรรม
- **สรุป/ผลลัพธ์:** ผู้อ่านมีกรอบเปรียบเทียบ style/pattern
- **เชื่อมโยง:** ไปสู่เล่ม 2–4

บทที่ 5 Architectural Drivers: Building Blocks for Decision Making

- **เรื่อง:** ตัวขับเคลื่อนการตัดสินใจทางสถาปัตยกรรม
- **วัตถุประสงค์:** ทำให้การตัดสินใจอิงความต้องการธุรกิจและคุณภาพระบบ
- **เป้าหมาย:** จัดลำดับ drivers และใช้เป็นเกณฑ์ตัดสินใจได้
- **หัวข้อหลัก:**
  - Functional requirements vs quality attributes
  - Business drivers
  - Technical drivers
  - การจัดลำดับความสำคัญ
  - การเชื่อม drivers กับ decision record
- **ข้อดีของบท:** ทำให้การเลือกสถาปัตยกรรมตรวจสอบย้อนกลับได้
- **ข้อเสีย/ข้อจำกัด:** ต้องมีผู้มีส่วนได้ส่วนเสียร่วมให้ข้อมูล
- **ข้อควรระวัง:** ทุกอย่างเป็น priority สูงไม่ได้
- **ตัวอย่าง:** ระบบที่ต้องรองรับผู้ใช้จำนวนมากแต่ทีมเล็ก
- **การประยุกต์ใช้:** ทำ Architecture Driver List ก่อนเลือก style
- **สรุป/ผลลัพธ์:** ผู้อ่านจัดลำดับ drivers ได้
- **เชื่อมโยง:** ไปสู่บทที่ 6–10 และเล่ม 5–7

ส่วนที่ 2: รูปแบบสถาปัตยกรรมและการจัดองค์ประกอบ

บทที่ 6 Monolithic Architecture

- **เรื่อง:** สถาปัตยกรรมแบบ monolith
- **วัตถุประสงค์:** เข้าใจจุดแข็ง จุดอ่อน และบริบทที่เหมาะสมของ monolith
- **เป้าหมาย:** ประเมินได้ว่าเมื่อใดควรใช้หรือไม่ใช้ monolith
- **หัวข้อหลัก:**
  - โครงสร้างโดยทั่วไป
  - ข้อดี: เริ่มต้นง่าย, deploy ง่าย
  - ข้อเสีย: coupling, scaling ทั้งก้อน
  - Modular monolith
  - เมื่อใดควรใช้
- **ข้อดีของบท:** ลดอคติว่า monolith เป็นสิ่งเลวร้ายเสมอ
- **ข้อเสีย/ข้อจำกัด:** ไม่ได้ลงลึกถึง microservices
- **ข้อควรระวัง:** monolith ที่ไม่มี modularity จะแก้ยาก
- **ตัวอย่าง:** ระบบ ERP ขนาดเล็ก
- **การประยุกต์ใช้:** ใช้ประเมินทางเลือกก่อนแยกบริการ
- **สรุป/ผลลัพธ์:** ผู้อ่านประเมินความเหมาะสมของ monolith ได้
- **เชื่อมโยง:** ไปสู่บทที่ 7 และเล่ม 2

บทที่ 7 Component Based Architecture

- **เรื่อง:** การออกแบบสถาปัตยกรรมด้วย component
- **วัตถุประสงค์:** เข้าใจ component, boundary และ contract
- **เป้าหมาย:** วาด component diagram เบื้องต้นได้
- **หัวข้อหลัก:**
  - Component คืออะไร
  - Boundary และ contract
  - Components vs classes
  - การสื่อสารระหว่าง component
  - การกำหนดความรับผิดชอบ
- **ข้อดีของบท:** เป็นฐานให้เข้าใจ microservices และ modularity
- **ข้อเสีย/ข้อจำกัด:** การแบ่ง component ต้องอาศัยบริบทโดเมน
- **ข้อควรระวัง:** component ใหญ่เกินไปหรือเล็กเกินไป
- **ตัวอย่าง:** ระบบ e-commerce แยก catalog, order, payment
- **การประยุกต์ใช้:** ใช้ทำ component map ของระบบ
- **สรุป/ผลลัพธ์:** ผู้อ่านกำหนด component และ boundary ได้
- **เชื่อมโยง:** ไปสู่บทที่ 14, 17 และเล่ม 2–3

บทที่ 8 PresentationDomainDataLayering

- **เรื่อง:** การแบ่งชั้นแบบ Presentation–Domain–Data
- **วัตถุประสงค์:** เข้าใจการแบ่ง layer พื้นฐานและ dependency
- **เป้าหมาย:** ออกแบบ layer เบื้องต้นได้อย่างมีเหตุผล
- **หัวข้อหลัก:**
  - แต่ละชั้นรับผิดชอบอะไร
  - Dependency ระหว่างชั้น
  - ข้อดี/ข้อเสีย
  - Domain layer ที่มี business logic
  - การทดสอบแต่ละชั้น
- **ข้อดีของบท:** เป็นพื้นฐานของ layered architecture
- **ข้อเสีย/ข้อจำกัด:** อาจไม่พอสำหรับโดเมนซับซ้อน
- **ข้อควรระวัง:** domain กลายเป็นชั้นบาง ๆ ที่ไม่มี logic
- **ตัวอย่าง:** Web app ที่แยก UI, business logic, data access
- **การประยุกต์ใช้:** ใช้ตรวจโครงสร้างโปรเจกต์ขนาดเล็ก
- **สรุป/ผลลัพธ์:** ผู้อ่านแยกหน้าที่ของ layer ได้
- **เชื่อมโยง:** ไปสู่บทที่ 9–10 และเล่ม 4

บทที่ 9 Layers, Hexagons, Features and Components

- **เรื่อง:** ทางเลือกในการจัดโครงสร้างนอกเหนือจาก layer
- **วัตถุประสงค์:** เปรียบเทียบแนวทางจัดองค์ประกอบหลายแบบ
- **เป้าหมาย:** เลือกแนวทางจัดโครงสร้างให้เหมาะกับบริบท

- **หัวข้อหลัก:**
  - Layer-based
  - Hexagonal เบื้องต้น
  - Feature-based
  - Component-based
  - เปรียบเทียบข้อดี/ข้อเสีย
- **ข้อดีของบท:** ทำให้เห็นทางเลือกมากกว่าการแบ่งตาม layer
- **ข้อเสีย/ข้อจำกัด:** เป็นภาพรวม ไม่ใช่คู่มือ implementation
- **ข้อควรระวัง:** การจัดตาม layer อาจซ่อน domain
- **ตัวอย่าง:** โครงสร้าง package แบบต่าง ๆ ของระบบเดียวกัน
- **การประยุกต์ใช้:** ใช้เลือกโครงสร้างโปรเจกต์ใหม่
- **สรุป/ผลลัพธ์:** ผู้อ่านเลือกแนวทางจัดโครงสร้างได้
- **เชื่อมโยง:** ไปสู่บทที่ 10, 18 และเล่ม 3–4

บทที่ 10 More than Concentric Layers

- **เรื่อง:** ข้อจำกัดของ concentric layers และสะพานไปสู่โดเมน
- **วัตถุประสงค์:** เชื่อมแนวคิด layer สู่ DDD และ Clean Architecture
- **เป้าหมาย:** เข้าใจว่าทำไมต้องสะท้อนโดเมนในโครงสร้าง
- **หัวข้อหลัก:**
  - ข้อจำกัดของ concentric layers
  - ทำไมต้องสะท้อนโดเมน
  - บทนำไปสู่ DDD/Clean Architecture
  - Dependency rule เบื้องต้น
- **ข้อดีของบท:** เป็นบทเชื่อมสำคัญไปเล่ม 3–4
- **ข้อเสีย/ข้อจำกัด:** ยังไม่ลงลึกเรื่อง DDD
- **ข้อควรระวัง:** อย่าแยกชั้นจน business logic กระจัดกระจาย
- **ตัวอย่าง:** ระบบที่มี business rules ซับซ้อน
- **การประยุกต์ใช้:** ใช้ตั้งคำถามก่อน refactor สถาปัตยกรรม
- **สรุป/ผลลัพธ์:** ผู้อ่านเห็นเส้นทางไปสู่สถาปัตยกรรมที่อิงโดเมน
- **เชื่อมโยง:** ไปสู่เล่ม 3 และเล่ม 4

ส่วนที่ 3: จากสถาปัตยกรรมสู่โค้ด

บทที่ 11 Software Architecture vs Code

- **เรื่อง:** ความสัมพันธ์ระหว่างสถาปัตยกรรมกับโค้ด
- **วัตถุประสงค์:** ทำให้เห็นว่าสถาปัตยกรรมปรากฏในโค้ดอย่างไร
- **เป้าหมาย:** ตรวจสอบ drift ระหว่าง design กับ code ได้
- **หัวข้อหลัก:**
  - สถาปัตยกรรมปรากฏในโค้ดอย่างไร
  - Drift ระหว่าง design กับ code
  - วิธีตรวจสอบ

  - สัญญาณเตือน
- **ข้อดีของบท:** เชื่อมเอกสารกับงานจริง
- **ข้อเสีย/ข้อจำกัด:** ต้องอาศัยตัวอย่างโค้ดประกอบ
- **ข้อควรระวัง:** เอกสารกับโค้ดไม่ตรงกัน
- **ตัวอย่าง:** package ที่ไม่ตรงกับ component diagram
- **การประยุกต์ใช้:** ใช้ใน architecture review
- **สรุป/ผลลัพธ์:** ผู้อ่านตรวจ alignment ได้
- **เชื่อมโยง:** ไปสู่บทที่ 12–13, 19

บทที่ 12 Mapping Software Architecture to Code

- **เรื่อง:** การแปลงสถาปัตยกรรมเป็นโครงสร้างโค้ด
- **วัตถุประสงค์:** ให้ผู้อ่าน map component สู่ package/module
- **เป้าหมาย:** ทำ architecture-to-code map ได้
- **หัวข้อหลัก:**
  - Component → package/module
  - Interface/contract
  - Dependency direction
  - การตั้งชื่อ
  - ตัวอย่าง mapping
- **ข้อดีของบท:** ทำให้สถาปัตยกรรมนำไปใช้ได้จริง
- **ข้อเสีย/ข้อจำกัด:** mapping 1:1 อาจไม่เหมาะทุกกรณี
- **ข้อควรระวัง:** mapping ที่ซับซ้อนเกินจำเป็น
- **ตัวอย่าง:** mapping component diagram สู่ package
- **การประยุกต์ใช้:** ใช้ตั้งโครงโปรเจกต์ใหม่หรือ refactor
- **สรุป/ผลลัพธ์:** ผู้อ่านสร้าง mapping ได้
- **เชื่อมโยง:** ไปสู่บทที่ 13, 17–18

บทที่ 13 An Architecturally-Evident Coding Style

- **เรื่อง:** สไตล์โค้ดที่ทำให้เห็นสถาปัตยกรรม
- **วัตถุประสงค์:** กำหนดแนวทางเขียนโค้ดที่สะท้อนสถาปัตยกรรม
- **เป้าหมาย:** ตั้ง coding style ที่ตรวจสอบ alignment ได้
- **หัวข้อหลัก:**
  - ชื่อ package สื่อความหมาย
  - การจัดกลุ่มไฟล์
  - Dependency ที่มองเห็นได้
  - สัญญาณเตือนเมื่อโค้ดเบี่ยงจากสถาปัตยกรรม
- **ข้อดีของบท:** ทำให้โค้ดสื่อสารสถาปัตยกรรมได้ด้วยตัวเอง
- **ข้อเสีย/ข้อจำกัด:** ต้องมีวินัยและการレビュー
- **ข้อควรระวัง:** ชื่อสวยแต่โครงสร้างไม่จริง
- **ตัวอย่าง:** โค้ดที่อ่านแล้วเห็น layer/component
- **การประยุกต์ใช้:** ใช้เป็น guideline ของทีม

- **สรุป/ผลลัพธ์:** ผู้อ่านกำหนด coding style ได้
- **เชื่อมโยง:** ไปสู่บทที่ 14–15, 19

บทที่ 14 Components vs Classes

- **เรื่อง:** ความแตกต่างระหว่างหน่วยโค้ดและหน่วยสถาปัตยกรรม
- **วัตถุประสงค์:** แยกบทบาทของ class และ component
- **เป้าหมาย:** ออกแบบ component โดยไม่สับสนกับ class
- **หัวข้อหลัก:**
  - Class คือหน่วยโค้ด
  - Component คือหน่วยสถาปัตยกรรม
  - ความสัมพันธ์และการแบ่งหน้าที่
  - Boundary ของ component
- **ข้อดีของบท:** ลดความสับสนในการออกแบบ
- **ข้อเสีย/ข้อจำกัด:** ต้องมีตัวอย่างประกอบ
- **ข้อควรระวัง:** ใช้ class แทน component จน boundary หาย
- **ตัวอย่าง:** class หลายตัวรวมเป็น component เดียว
- **การประยุกต์ใช้:** ใช้ตรวจ component map
- **สรุป/ผลลัพธ์:** ผู้อ่านแยกหน่วยโค้ดกับหน่วยสถาปัตยกรรมได้
- **เชื่อมโยง:** ไปสู่บทที่ 7, 17

บทที่ 15 Reflecting Architecture and Domain in Code

- **เรื่อง:** ให้โค้ดสะท้อนทั้งสถาปัตยกรรมและโดเมน
- **วัตถุประสงค์:** รวมแนวคิด architecture และ domain ในโครงสร้างโค้ด
- **เป้าหมาย:** ออกแบบโครงสร้างโค้ดที่สื่อความหมายได้
- **หัวข้อหลัก:**
  - โครงสร้างที่สื่อ domain
  - โครงสร้างที่สื่อ architecture
  - การหลีกเลี่ยงโครงสร้างตาม framework
  - การรักษา dependency rule
- **ข้อดีของบท:** เป็นสะพานไป DDD และ Clean Architecture
- **ข้อเสีย/ข้อจำกัด:** ต้องเข้าใจโดเมนพอสมควร
- **ข้อควรระวัง:** พยายามสะท้อนทุกอย่างจนซับซ้อนเกิน
- **ตัวอย่าง:** package ตาม domain + component
- **การประยุกต์ใช้:** ใช้ปรับโครงสร้างก่อนทำ DDD
- **สรุป/ผลลัพธ์:** ผู้ออกแบบโครงสร้างโค้ดที่สื่อความหมายได้
- **เชื่อมโยง:** ไปสู่เล่ม 3–4

ส่วนที่ 4: Modularity, Packaging และการทดสอบ

บทที่ 16 Modularity and Testability

- **เรื่อง:** ความเชื่อมโยงระหว่าง modularity กับการทดสอบ
- **วัตถุประสงค์:** ทำให้ผู้อ่านเห็นว่าการออกแบบที่ดีทดสอบง่ายกว่า
- **เป้าหมาย:** ประเมิน testability ของสถาปัตยกรรมได้
- **หัวข้อหลัก:**
  - Coupling/cohesion
  - Test pyramid เบื้องต้น
  - การทดสอบ component
  - Dependency injection เบื้องต้น
- **ข้อดีของบท:** เชื่อมสถาปัตยกรรมกับคุณภาพการทดสอบ
- **ข้อเสีย/ข้อจำกัด:** ไม่ได้ลงลึกเครื่องมือทดสอบ
- **ข้อควรระวัง:** modularity ที่ไม่จริงทำให้ทดสอบยาก
- **ตัวอย่าง:** component ที่ทดสอบง่าย vs ยาก
- **การประยุกต์ใช้:** ใช้ประเมินก่อน refactor
- **สรุป/ผลลัพธ์:** ผู้อ่านประเมิน testability ได้
- **เชื่อมโยง:** ไปสู่บทที่ 17 และเล่ม 4

บทที่ 17 Package by Component and Architecturally-Aligned Testing

- **เรื่อง:** จัด package ตาม component และทดสอบให้สอดคล้อง
- **วัตถุประสงค์:** เชื่อมโครงสร้าง package กับกลยุทธ์การทดสอบ
- **เป้าหมาย:** ออกแบบ package และ test strategy ที่สอดคล้องกัน
- **หัวข้อหลัก:**
  - Package by component
  - Testing ตาม boundary
  - Contract test เบื้องต้น
  - การป้องกันการทดสอบข้าม boundary
- **ข้อดีของบท:** ทำให้การทดสอบสะท้อนสถาปัตยกรรม
- **ข้อเสีย/ข้อจำกัด:** ต้องมีความรู้เรื่อง test strategy
- **ข้อควรระวัง:** ทดสอบข้าม boundary โดยไม่ตั้งใจ
- **ตัวอย่าง:** package structure และชุดทดสอบ
- **การประยุกต์ใช้:** ใช้ตั้งโครงทดสอบโปรเจกต์ใหม่
- **สรุป/ผลลัพธ์:** ผู้ออกแบบ package และ test strategy ได้
- **เชื่อมโยง:** ไปสู่บทที่ 18–19 และเล่ม 4

บทที่ 18 Schools of Package Architecture – An Illustration

- **เรื่อง:** แนวทางจัด package หลายสำนัก
- **วัตถุประสงค์:** เปรียบเทียบแนวทาง package architecture
- **เป้าหมาย:** เลือกแนวทาง package ได้อย่างมีเหตุผล
- **หัวข้อหลัก:**
  - Package by layer

  - Package by feature
  - Package by component
  - Package by domain
  - ข้อดี/ข้อเสียแต่ละแบบ
- **ข้อดีของบท:** ทำให้เห็นทางเลือกและผลกระทบ
- **ข้อเสีย/ข้อจำกัด:** ไม่มีแบบเดียวที่เหมาะทุกทีม
- **ข้อควรระวัง:** เลือกตามกระแสโดยไม่ดูบริบท
- **ตัวอย่าง:** ระบบเดียวกันจัด package 4 แบบ
- **การประยุกต์ใช้:** ใช้ workshop เลือกโครงสร้าง package
- **สรุป/ผลลัพธ์:** ผู้อ่านเลือกแนวทาง package ได้
- **เชื่อมโยง:** ไปสู่บทที่ 19 และเล่ม 3–4

บทที่ 19 Aligning Software Architecture and Code

- **เรื่อง:** ทำให้สถาปัตยกรรมและโค้ดสอดคล้องกันอย่างต่อเนื่อง
- **วัตถุประสงค์:** วางกลไกกำกับ alignment
- **เป้าหมาย:** ตรวจและรักษาความสอดคล้องได้
- **หัวข้อหลัก:**
  - Architecture review
  - Fitness function เบื้องต้น
  - Static analysis
  - Refactoring เพื่อรักษา alignment
- **ข้อดีของบท:** ทำให้ alignment ไม่ใช่เรื่องครั้งเดียว
- **ข้อเสีย/ข้อจำกัด:** ต้องมีเครื่องมือและวินัยทีม
- **ข้อควรระวัง:** ปล่อย drift สะสมจนแก้ยาก
- **ตัวอย่าง:** ตรวจ dependency rule ด้วยเครื่องมือ
- **การประยุกต์ใช้:** ใช้ตั้งกลไกตรวจสอบใน CI
- **สรุป/ผลลัพธ์:** ผู้อ่านวางกลไกกำกับ alignment ได้
- **เชื่อมโยง:** ไปสู่บทที่ 20 และเล่ม 7

ส่วนที่ 5: การ dokument และกรณีศึกษา

บทที่ 20 Documenting Software Architecture

- **เรื่อง:** การ dokument สถาปัตยกรรมอย่างพอดี
- **วัตถุประสงค์:** ให้ผู้อ่าน dokument อย่างมีวัตถุประสงค์และใช้งานได้
- **เป้าหมาย:** จัดทำเอกสารสถาปัตยกรรมขั้นต่ำได้
- **หัวข้อหลัก:**
  - Document อะไร
  - สำหรับใคร
  - Context diagram, component diagram, deployment diagram
  - Architecture Decision Record (ADR) เบื้องต้น
- **ข้อดีของบท:** ลดเอกสารเกินจำเป็นและเพิ่มการสื่อสาร

- **ข้อเสีย/ข้อจำกัด:** ต้องปรับตามผู้ชม
- **ข้อควรระวัง:** เอกสารมากเกินไปแต่ไม่ถูกใช้
- **ตัวอย่าง:** ชุดเอกสารสถาปัตยกรรมขนาดเล็ก
- **การประยุกต์ใช้:** ใช้ตั้ง template เอกสารของทีม
- **สรุป/ผลลัพธ์:** ผู้อ่านจัดทำเอกสารขั้นต่ำได้
- **เชื่อมโยง:** ไปสู่บทที่ 21–22

บทที่ 21 Simple Sketches for Diagramming your Software Architecture

- **เรื่อง:** การวาด diagram อย่างง่ายแต่สื่อสารได้
- **วัตถุประสงค์:** ให้ผู้อ่านวาด diagram เพื่อสื่อสารสถาปัตยกรรม
- **เป้าหมาย:** วาด context/component diagram เบื้องต้นได้
- **หัวข้อหลัก:**
  - สัญลักษณ์พื้นฐาน
  - Context → Container → Component
  - การแสดง dependency
  - การหลีกเลี่ยง diagram ที่รก
- **ข้อดีของบท:** ใช้งานได้ทันทีในทีม
- **ข้อเสีย/ข้อจำกัด:** ไม่ได้ครอบคลุม notation ทั้งหมด
- **ข้อควรระวัง:** diagram ไม่มี legend หรือขอบเขต
- **ตัวอย่าง:** simple sketch ของระบบหนึ่ง
- **การประยุกต์ใช้:** ใช้ใน workshop และเอกสาร
- **สรุป/ผลลัพธ์:** ผู้อ่านวาด diagram เพื่อสื่อสารได้
- **เชื่อมโยง:** ไปสู่บทที่ 22 และเล่ม 2–4

บทที่ 22 Case Study: techtribes.je – Context

- **เรื่อง:** กรณีศึกษาการทำความเข้าใจ context
- **วัตถุประสงค์:** ประยุกต์แนวคิด context และ drivers กับกรณีศึกษา
- **เป้าหมาย:** เขียน context diagram และ driver list ได้
- **หัวข้อหลัก:**
  - System context
  - ผู้ใช้และระบบภายนอก
  - Constraints
  - Architectural drivers
  - บทเรียนจากกรณีศึกษา
- **ข้อดีของบท:** เชื่อมแนวคิดทั้งหมดกับตัวอย่างจริง
- **ข้อเสีย/ข้อจำกัด:** เป็นกรณีศึกษาเดียว
- **ข้อควรระวัง:** มองข้ามระบบภายนอกและผู้ใช้งาน
- **ตัวอย่าง:** techtribes.je
- **การประยุกต์ใช้:** ใช้เป็นแบบฝึกหัดวิเคราะห์ระบบ
- **สรุป/ผลลัพธ์:** ผู้อ่านวิเคราะห์ context ของระบบได้
- **เชื่อมโยง:** ไปสู่บทที่ 23 และเล่ม 2–7

```
---

### บทที่ 23 การประยุกต์ใช้และสรุปเล่ม

- **เรื่อง:** การนำเนื้อหาทั้งเล่มไปใช้
- **วัตถุประสงค์:** สรุปและแปลงความรู้เป็นแผนปฏิบัติ
- **เป้าหมาย:** มีแผนประยุกต์ใช้ในองค์กร
- **หัวข้อหลัก:**
  - Workshop: Architecture Context Canvas
  - Workshop: Style/Pattern Decision
  - Workshop: Package Alignment Review
  - สรุปสาระสำคัญ
  - เชื่อมไปเล่ม 2–4
- **ข้อดีของบท:** ปิดเล่มด้วยการลงมือทำ
- **ข้อเสีย/ข้อจำกัด:** ต้องปรับตามบริบทองค์กร
- **ข้อควรระวัง:** อย่านำไปใช้แบบคัดลอกโดยไม่ปรับบริบท
- **ตัวอย่าง:** แผนนำไปใช้ใน 30 วัน
- **การประยุกต์ใช้:** ใช้เป็นแผนดำเนินงานหลังอ่านจบ
- **สรุป/ผลลัพธ์:** ผู้อ่านมีแผนประยุกต์ใช้
- **เชื่อมโยง:** ไปสู่เล่ม 2–7

---

# ภาคผนวก

## ภาคผนวก A: แม่แบบ Architecture Context Canvas

| หัวข้อ | คำอธิบาย |
|---|---|
| ระบบคืออะไร | วัตถุประสงค์และขอบเขต |
| ผู้ใช้/actor | ใครใช้งานหรือเกี่ยวข้อง |
| ระบบภายนอก | ระบบที่เชื่อมต่อ |
| Constraints | ข้อจำกัดทางธุรกิจ/เทคนิค/กฎหมาย |
| Quality attributes | คุณภาพที่สำคัญ |
| Architectural drivers | ปัจจัยขับเคลื่อนการตัดสินใจ |
| ความเสี่ยงเบื้องต้น | ความเสี่ยงที่มองเห็น |

## ภาคผนวก B: แม่แบบ Style/Pattern Decision Record

| หัวข้อ | รายละเอียด |
|---|---|
| ปัญหา | ปัญหาที่ต้องตัดสินใจ |
| ทางเลือก | ทางเลือกที่พิจารณา |
| เหตุผล | เหตุผลประกอบ |
| ข้อดี/ข้อเสีย | ของแต่ละทางเลือก |
| ความเสี่ยง | ความเสี่ยงและมาตรการ |
```

```
| การตัดสินใจ | ทางเลือกที่เลือก |
| ผู้รับผิดชอบ | ผู้ตัดสินใจ |
| วันที่ | วันที่ตัดสินใจ |

## ภาคผนวก C: แม่แบบ Component & Dependency Map

| Component | ความรับผิดชอบ | Interface/Contract | Dependency | Owner |
|---|---|---|---|---|
| | | | | |

## ภาคผนวก D: Package Alignment Checklist

- [ ] Package สื่อ component/domain หรือไม่
- [ ] Dependency direction ถูกต้องหรือไม่
- [ ] มี cyclic dependency หรือไม่
- [ ] Test ตรง boundary หรือไม่
- [ ] ชื่อ package สอดคล้องกับสถาปัตยกรรมหรือไม่
- [ ] มี package ที่ใหญ่หรือเล็กเกินไปหรือไม่

## ภาคผนวก E: Diagram Checklist

- [ ] มีชื่อ diagram
- [ ] มี legend
- [ ] มีขอบเขต
- [ ] มีทิศทาง dependency
- [ ] มีคำอธิบายประกอบ
- [ ] ไม่รกเกินไป
- [ ] ตรงกับโค้ดหรือเอกสารล่าสุด

## ภาคผนวก F: Glossary กลาง

| คำ | ความหมายโดยย่อ |
|---|---|
| Software Architecture | โครงสร้างหลักและหลักการของระบบ |
| Architectural Style | แนวทางจัดโครงสร้างระดับสูง |
| Architectural Pattern | รูปแบบการแก้ปัญหาเชิงสถาปัตยกรรม |
| Design Pattern | รูปแบบการแก้ปัญหาเชิงออกแบบโค้ด |
| Component | หน่วยสถาปัตยกรรมที่มี boundary |
| Module/Package | หน่วยจัดกลุ่มโค้ด |
| Layer | ชั้นความรับผิดชอบ |
| Boundary | ขอบเขตความรับผิดชอบ |
| Quality Attribute | คุณภาพของระบบ เช่น scalability, maintainability |
| Architectural Driver | ปัจจัยขับเคลื่อนการตัดสินใจ |
| ADR | Architecture Decision Record |
| Alignment | ความสอดคล้องระหว่างสถาปัตยกรรมกับโค้ด |

---
```

เกณฑ์ประเมินความสำเร็จของเล่ม

เกณฑ์	ตัวชี้วัด
ความเข้าใจ	อธิบาย style/pattern/design pattern ต่างกันได้
การนำไปใช้	มี diagram และ package map ของระบบตนเอง
การตัดสินใจ	มี decision record อย่างน้อย 1 ฉบับ
ความสอดคล้อง	ตรวจ alignment ระหว่างสถาปัตยกรรมกับโค้ดได้
การส่งต่อ	เข้าใจว่าบทใดเชื่อมไปเล่ม 2, 3, 4

คู่มือเล่ม 1: ปฐมบทสถาปัตยกรรมซอฟต์แวร์
Software Architecture Foundations

> ชุดคู่มือสถาปัตยกรรมซอฟต์แวร์และการบริหารโครงการ
> เล่ม 1 จาก 7 เล่ม
> ฉบับสมบูรณ์ v1.0

คำนำ

สถาปัตยกรรมซอฟต์แวร์เป็นเรื่องที่อยู่รอบตัวเราทุกวัน ตั้งแต่การตัดสินใจว่าโค้ดควรอยู่ใน package ใด ไปจนถึงการเลือกว่าจะแยกบริการ

คู่มือเล่มนี้จึงถูกออกแบบมาเพื่อสร้างภาษากลางนั้น ตั้งแต่คำนิยามพื้นฐาน การแยกแยะ style/pattern/design pattern ไปจนถึงกา

เมื่ออ่านจบ ผู้อ่านควรสามารถอธิบายสถาปัตยกรรมของระบบตนเอง วาด diagram ที่สื่อสารได้ เลือก style/pattern อย่างมีเหตุผล

ข้อมูลเอกสาร

รายการ	รายละเอียด
ชื่อเล่ม	ปฐมบทสถาปัตยกรรมซอฟต์แวร์
ชื่อรอง	พื้นฐาน ภาษากลาง และการเชื่อมสถาปัตยกรรมสู่โค้ด
กลุ่มผู้อ่าน	Software Architect, Tech Lead, Senior Developer, Tester, DevOps, PM
ระดับ	พื้นฐานถึงกลาง
โครงสร้างหลัก	เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป
เล่มที่เชื่อมโยง	เล่ม 2 Microservices, เล่ม 3 DDD, เล่ม 4 Clean Architecture, เล่ม 5 ความเสี่ยง, เล่ม 6 POC, เล่
จำนวนบท	23 บท + 6 ภาคผนวก

วิธีใช้เอกสาร

1. **บทที่ 1–5** ใช้สร้างภาษากลางและความเข้าใจพื้นฐาน — อ่านก่อนทุกบท
2. **บทที่ 6–10** ใช้เลือกและเปรียบเทียบรูปแบบสถาปัตยกรรม — อ่านก่อนตัดสินใจเชิงโครงสร้าง
3. **บทที่ 11–15** ใช้เชื่อมสถาปัตยกรรมลงสู่โค้ด — อ่านก่อน refactor หรือตั้งโปรเจกต์ใหม่
4. **บทที่ 16–19** ใช้ตรวจสอบ modularity, package และการทดสอบ — อ่านก่อนวาง test strategy
5. **บทที่ 20–23** ใช้ dokument, วาด diagram และนำไปประยุกต์ใช้ — อ่านก่อนทำ architecture review
6. **ภาคผนวก** ใช้เป็นเครื่องมือประกอบ workshop และการทำงานประจำวัน

ส่วนที่ 1: ภาพรวมและภาษากลางของสถาปัตยกรรม

บทที่ 1: The Software Architecture Chronicles

1.1 เรื่อง
สถาปัตยกรรมซอฟต์แวร์คืออะไร ทำไมต้องให้ความสำคัญ และบทความชุด *The Software Architecture Chronicles* พยายามต

1.2 วัตถุประสงค์
- สร้างคำนิยามร่วมของ "สถาปัตยกรรมซอฟต์แวร์"
- อธิบายบทบาทของ architect และความสัมพันธ์กับทีม
- ทำให้เห็นว่าสถาปัตยกรรมไม่ใช่แค่ diagram หรือเทคโนโลยี

1.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- อธิบายความหมายของสถาปัตยกรรมซอฟต์แวร์ด้วยคำของตนเอง
- ระบุได้ว่าสถาปัตยกรรมเกี่ยวข้องกับใครบ้าง
- เห็นความเชื่อมโยงระหว่างสถาปัตยกรรม ธุรกิจ และโค้ด

1.4 หัวข้อหลัก

1.4.1 ความหมายของ Software Architecture
สถาปัตยกรรมซอฟต์แวร์คือ **โครงสร้างหลักของระบบ** ที่ประกอบด้วย:
- **องค์ประกอบ (components)** — ระบบแบ่งเป็นส่วนอะไรบ้าง
- **ความสัมพันธ์ (relationships)** — ส่วนต่าง ๆ เชื่อมต่อกันอย่างไร
- **หลักการและข้อจำกัด (principles & constraints)** — อะไรทำได้ อะไรทำไม่ได้
- **เหตุผล (rationale)** — ทำไมจึงเลือกโครงสร้างนี้

สถาปัตยกรรมไม่ใช่แค่ "เทคโนโลยีที่ใช้" แต่เป็นการตัดสินใจที่ส่งผลระยะยาวต่อความสามารถในการเปลี่ยนแปลงระบบ

1.4.2 บทบาทของ Architect
Architect ไม่ใช่ "คนวาด diagram" แต่คือคนที่:
- **เชื่อมธุรกิจกับเทคนิค** — แปลความต้องการเป็นโครงสร้าง
- **ตัดสินใจภายใต้ความไม่แน่นอน** — เลือกทางที่สมเหตุสมผลที่สุด ณ ขณะนั้น
- **สื่อสารกับผู้มีส่วนได้ส่วนเสีย** — ทั้ง developer, PM, business, operations
- **ดูแล alignment** — ทำให้สถาปัตยกรรมกับโค้ดไม่แยกจากกัน

- **บริหาร trade-offs** — ไม่มีทางเลือกที่ดีทุกด้าน

1.4.3 สถาปัตยกรรมกับความต้องการทางธุรกิจ

สถาปัตยกรรมที่ดีต้องตอบคำถามธุรกิจได้:

- ระบบต้องรองรับผู้ใช้เท่าไร
- ต้องเปลี่ยนแปลงบ่อยแค่ไหน
- มีข้อจำกัดด้านกฎหมายหรือไม่
- ทีมมีขนาดและทักษะอย่างไร
- งบประมาณและเวลามีเท่าไร

1.4.4 วงจรชีวิตของสถาปัตยกรรม

สถาปัตยกรรมไม่ได้ถูกกำหนดครั้งเดียวแล้วจบ แต่มีวงจร:

1. **เข้าใจบริบท** — ธุรกิจ ผู้ใช้ ข้อจำกัด
2. **กำหนด drivers** — ปัจจัยขับเคลื่อนการตัดสินใจ
3. **ออกแบบทางเลือก** — สร้างตัวเลือกหลายแบบ
4. **ตัดสินใจและบันทึก** — เลือกและทำ decision record
5. **นำไปใช้** — สื่อสารและลงมือทำ
6. **ตรวจสอบและปรับ** — ตรวจ alignment และปรับเมื่อบริบทเปลี่ยน

1.4.5 สถาปัตยกรรมกับโค้ดและทีม

สถาปัตยกรรมที่ดีต้อง:

- **ปรากฏในโค้ด** — โครงสร้าง package สะท้อนสถาปัตยกรรม
- **ทีมเข้าใจร่วมกัน** — ไม่ใช่ความรู้ของ architect คนเดียว
- **ทดสอบได้** — boundary ชัดเจนทำให้ทดสอบง่าย
- **เปลี่ยนแปลงได้** — ไม่ผูกติดกับเทคโนโลยีเดียว

1.5 ข้อดีของบท

- เป็นจุดเริ่มต้นที่สร้างภาษากลางให้ทั้งเล่ม
- ลดความสับสนเรื่องคำนิยาม
- ทำให้เห็นภาพรวมก่อนลงรายละเอียด

1.6 ข้อเสีย/ข้อจำกัด

- ยังไม่ลงรายละเอียดเชิงเทคนิค
- ตัวอย่างเป็นภาพรวม ต้องอ่านบทถัดไปเพื่อความเข้าใจลึก

1.7 ข้อควรระวัง

- อย่านิยามสถาปัตยกรรมเพียงเรื่องเทคโนโลยีหรือ diagram
- อย่าคิดว่าสถาปัตยกรรมเป็นงานของ architect คนเดียว
- อย่าละเลยเหตุผล (rationale) เบื้องหลังการตัดสินใจ

1.8 ตัวอย่าง

บริษัทหนึ่งเริ่มต้นด้วยระบบ monolith ขนาดเล็ก ทีม 3 คน ต่อมาธุรกิจเติบโต ต้องรองรับผู้ใช้ 10 เท่า ทีมขยายเป็น 3 ทีม สถาปัตยก

1.9 การประยุกต์ใช้

- ใช้เปิดประชุม architecture review เพื่อตั้งคำนิยามร่วม
- ใช้ตั้งคำถามว่า "สถาปัตยกรรมของเราคืออะไร" ในทีม
- ใช้เป็น checklist ว่าการตัดสินใจมี rationale รองรับหรือไม่

1.10 สรุป/ผลลัพธ์
ผู้อ่านมีภาษากลางเบื้องต้นเกี่ยวกับสถาปัตยกรรม เข้าใจว่าสถาปัตยกรรมคือโครงสร้าง+หลักการ+เหตุผล ไม่ใช่แค่เทคโนโลยี และเห็นวงจร

1.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 2–4 เพื่อลงรายละเอียดเรื่อง premises และ patterns
- ไปสู่เล่ม 7 การบริหารโครงการ เพื่อเชื่อมสถาปัตยกรรมกับแผนงาน

บทที่ 2: Software Architecture Premises

2.1 เรื่อง
ข้อตั้งต้น สมมติฐาน และข้อจำกัดที่อยู่เบื้องหลังการตัดสินใจทางสถาปัตยกรรม

2.2 วัตถุประสงค์
- ทำให้ผู้อ่านตระหนักว่าทุกการตัดสินใจมีสมมติฐานรองรับ
- อธิบาย quality attributes, constraints และ trade-offs
- เชื่อม premises กับ architectural drivers

2.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ระบุ quality attributes ที่สำคัญของระบบได้
- ระบุ constraints ทางธุรกิจ เทคนิค และกฎหมายได้
- เข้าใจว่า trade-offs เป็นเรื่อง unavoidable

2.4 หัวข้อหลัก

2.4.1 Quality Attributes
Quality attributes คือคุณสมบัติที่ไม่ใช่ฟังก์ชันหลัก แต่สำคัญต่อความสำเร็จของระบบ:

หมวด	ตัวอย่าง
Performance	response time, throughput
Scalability	รองรับผู้ใช้เพิ่ม, horizontal/vertical
Availability	uptime, fault tolerance
Maintainability	อ่านโค้ดง่าย, แก้ไขง่าย
Testability	ทดสอบได้ครอบคลุม
Security	authentication, authorization, encryption
Usability	ใช้งานง่าย
Deployability	deploy บ่อยได้โดยไม่กระทบ
Observability	ตรวจสอบสถานะได้

Quality attributes มักขัดแย้งกัน เช่น scalability สูงอาจแลกกับ complexity ที่เพิ่มขึ้น

2.4.2 Constraints
Constraints คือข้อจำกัดที่ไม่สามารถละเลยได้:

- **ธุรกิจ** – งบประมาณ เวลา ทีม
- **เทคนิค** – ระบบเดิมที่ต้องเชื่อมต่อ, technology stack ที่องค์กรกำหนด
- **กฎหมาย/ข้อบังคับ** – PDPA, GDPR, มาตรฐานอุตสาหกรรม
- **องค์กร** – นโยบาย security, procurement

2.4.3 Architectural Drivers

Architectural drivers คือปัจจัยที่ขับเคลื่อนการตัดสินใจหลัก ๆ มักมาจาก:
- Quality attributes ที่สำคัญที่สุด 2–3 ตัว
- Constraints ที่ส่งผลใหญ่
- ความต้องการธุรกิจเชิงกลยุทธ์

2.4.4 Trade-offs

ทุกการตัดสินใจมี trade-off:
- เลือก microservices → scalability ดี แต่ complexity สูง
- เลือก monolith → เริ่มง่าย แต่ scaling ยาก
- เลือก NoSQL → flexible แต่ consistency อาจอ่อนกว่า

การยอมรับ trade-off อย่างชัดเจนดีกว่าการไม่ยอมรับว่ามี trade-off

2.4.5 สมมติฐานที่ต้องตรวจสอบ

สมมติฐานเช่น "ผู้ใช้จะไม่เกิน 10,000 คนในปีแรก" หรือ "ทีมจะโตเป็น 20 คน" ล้วนส่งผลต่อสถาปัตยกรรม ต้องบันทึกและตรวจสอบเ

2.5 ข้อดีของบท

- ทำให้การตัดสินใจสถาปัตยกรรมมีเหตุผลรองรับ
- ลดการตัดสินใจตามกระแสหรืออารมณ์
- เชื่อมสถาปัตยกรรมกับความต้องการธุรกิจอย่างชัดเจน

2.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยข้อมูลธุรกิจและเทคนิคประกอบ
- Quality attributes บางตัววัดยาก
- ต้องมีการทบทวนเป็นระยะ

2.7 ข้อควรระวัง

- สมมติฐานที่ไม่ถูกตรวจสอบจะกลายเป็นความเสี่ยง
- อย่าพยายาม optimize ทุก quality attribute พร้อมกัน
- อย่าละเลย constraints ที่ดูเหมือนเล็กน้อย

2.8 ตัวอย่าง

ทีมพัฒนาระบบ e-commerce ต้องเลือกระหว่าง SQL และ NoSQL:
- **Quality attributes ที่สำคัญ:** consistency ของข้อมูลคำสั่งซื้อ, scalability ช่วง flash sale
- **Constraints:** ทีมถนัด SQL, งบประมาณจำกัด
- **Trade-off:** เลือก SQL เพื่อ consistency และความถนัดทีม ยอมรับว่า scaling ต้องใช้ read replica + caching
- **สมมติฐาน:** flash sale จะเกิดขึ้นไม่เกิน 4 ครั้ง/ปี

2.9 การประยุกต์ใช้

- จัดทำรายการ quality attributes และจัดลำดับความสำคัญ
- บันทึก constraints และสมมติฐานก่อนออกแบบ

- ใช้เป็น input ของ Architecture Context Canvas (ภาคผนวก A)

2.10 สรุป/ผลลัพธ์
ผู้อ่านระบุ quality attributes, constraints, drivers และ trade-offs ของระบบได้ และเข้าใจว่าการตัดสินใจที่ดีต้องมี pr

2.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 5 (Architectural Drivers)
- ไปสู่เล่ม 5 (ความเสี่ยง) — สมมติฐานที่ไม่ตรวจสอบกลายเป็นความเสี่ยง
- ไปสู่เล่ม 6 (POC) — ใช้พิสูจน์สมมติฐาน

บทที่ 3: Architectural Styles vs. Architectural Patterns vs. Design Patterns

3.1 เรื่อง
การแยกความแตกต่างระหว่าง architectural style, architectural pattern และ design pattern ซึ่งมักใช้สับสน

3.2 วัตถุประสงค์
- สร้างความชัดเจนเรื่องระดับของแนวทางสถาปัตยกรรม
- ทำให้ทีมใช้คำศัพท์ตรงกัน
- ลดความสับสนในเอกสารและการสื่อสาร

3.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- จัดประเภทแนวทางสถาปัตยกรรมได้ถูกต้อง
- ระบุได้ว่าแนวทางหนึ่งเป็น style, pattern หรือ design pattern
- ใช้คำศัพท์อย่างแม่นยำในการสื่อสาร

3.4 หัวข้อหลัก

3.4.1 Architectural Style
Style คือแนวทางจัดโครงสร้างระดับสูงของระบบทั้งระบบ มักกำหนด:
- ประเภทขององค์ประกอบ
- วิธีการเชื่อมต่อ
- หลักการจัดวาง

ตัวอย่าง: Layered, Component-based, Monolithic, Event-driven, Microservices

Style มักเป็น " famille" ของ pattern ที่ใช้ร่วมกัน

3.4.2 Architectural Pattern
Pattern คือรูปแบบการแก้ปัญหาเชิงสถาปัตยกรรมที่เฉพาะเจาะจงกว่า style มักแก้ปัญหาในบริบทหนึ่ง ๆ

ตัวอย่าง: MVC, Hexagonal, CQRS, API Gateway, Circuit Breaker

Pattern อยู่ในระดับที่ลงรายละเอียดกว่า style แต่ยังไม่ถึงระดับโค้ด

3.4.3 Design Pattern

Design Pattern คือรูปแบบการแก้ปัญหาในระดับโค้ด/class

ตัวอย่าง: Strategy, Factory, Observer, Repository, Adapter

Design pattern ไม่ได้เปลี่ยนโครงสร้างระบบโดยรวม แต่ช่วยให้โค้ดภายในดีขึ้น

3.4.4 ระดับการนำไปใช้

ระดับ	หน่วย	ตัวอย่าง	ผลกระทบ
Style	ทั้งระบบ	Layered, Microservices	โครงสร้างโดยรวม
Architectural Pattern	กลุ่ม component	MVC, CQRS, Hexagonal	การจัดองค์ประกอบ
Design Pattern	class/function	Strategy, Factory	โค้ดภายใน

3.4.5 ตัวอย่างการจัดหมวด

- **Layered** = style
- **MVC** = architectural pattern (บางตำราจัดเป็น style)
- **Repository** = design pattern (บางครั้งใช้เป็น architectural pattern ใน DDD)
- **Clean Architecture** = architectural pattern/style ผสม

ความคลุมเครือนี้เป็นเรื่องปกติ สิ่งสำคัญคือเข้าใจระดับและผลกระทบ

3.5 ข้อดีของบท

- ลดความสับสนในทีมและเอกสาร
- ทำให้การเลือกใช้แนวทางมีระดับที่ชัดเจน
- เป็นพื้นฐานของบทที่ 4 และบทถัดไป

3.6 ข้อเสีย/ข้อจำกัด

- คำนิยามในอุตสาหกรรมอาจแตกต่างกันบ้าง
- บางแนวทางอยู่กึ่งกลางระหว่าง style กับ pattern

3.7 ข้อควรระวัง

- ใช้คำผิดระดับทำให้การตัดสินใจคลาดเคลื่อน
- อย่าเปรียบเทียบ style กับ design pattern โดยตรง
- อย่าคิดว่าการใช้ design pattern หลายตัวเท่ากับมีสถาปัตยกรรม

3.8 ตัวอย่าง

ทีมหนึ่งกล่าวว่า "เราใช้ Repository pattern เป็นสถาปัตยกรรม" – ที่จริง Repository เป็น design pattern ที่ใช้ภายใน lay

3.9 การประยุกต์ใช้

- ใช้ตรวจทาน glossary ของทีม
- ใช้ในการเขียน decision record ให้ระบุระดับให้ชัด
- ใช้สอนทีมใหม่เรื่องภาษากลาง

3.10 สรุป/ผลลัพธ์

ผู้อ่านแยกประเภทแนวทางสถาปัตยกรรมได้ และใช้คำศัพท์อย่างแม่นยำในการสื่อสาร

3.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 4 (รายละเอียด patterns/styles)
- ไปสู่เล่ม 4 (Clean Architecture เป็น architectural pattern)

บทที่ 4: Architectural Patterns and Styles

4.1 เรื่อง
ภาพรวมรูปแบบสถาปัตยกรรมที่พบบ่อยและบริบทการใช้งาน

4.2 วัตถุประสงค์
- แนะนำ style/pattern หลักที่ผู้อ่านควรรู้
- ทำให้เห็นข้อดี/ข้อเสียและบริบทที่เหมาะสม
- เป็นแผนที่นำทางก่อนลงลึกในเล่มถัดไป

4.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- อธิบาย style/pattern หลักได้
- เลือก style/pattern เบื้องต้นพร้อมเหตุผลได้
- รู้ว่าต้องไปอ่านเล่มใดต่อสำหรับแต่ละแนวทาง

4.4 หัวข้อหลัก

4.4.1 Layered Architecture
- แบ่งระบบเป็นชั้น (presentation, business, data)
- Dependency ไปทางเดียว
- **ข้อดี:** เข้าใจง่าย, แยกหน้าที่ชัด
- **ข้อเสีย:** อาจกลายเป็น "lasagna code" ที่ layer เยอะเกิน
- **เหมาะกับ:** ระบบทั่วไปที่ต้องการความชัดเจน

4.4.2 Component-Based Architecture
- แบ่งระบบเป็น component ที่มี boundary ชัด
- สื่อสารผ่าน interface/contract
- **ข้อดี:** modularity สูง, ทดสอบง่าย
- **ข้อเสีย:** ต้องออกแบบ boundary ดี
- **เหมาะกับ:** ระบบที่ต้องการแยกส่วนชัดเจน

4.4.3 Monolithic Architecture
- รวมทุกอย่างใน deployable unit เดียว
- **ข้อดี:** เริ่มง่าย, deploy ง่าย, debug ง่าย
- **ข้อเสีย:** coupling สูงเมื่อโต, scaling ทั้งก้อน
- **เหมาะกับ:** ระบบขนาดเล็ก, ทีมเล็ก, ช่วงเริ่มต้น

4.4.4 Event-Driven Architecture (เบื้องต้น)
- องค์ประกอบสื่อสารผ่าน event

- **ข้อดี:** decoupling, scalability
- **ข้อเสีย:** debug ยาก, eventual consistency
- **เหมาะกับ:** ระบบที่ต้องการ reactive/async

4.4.5 Microservices Architecture (เบื้องต้น)
- แยกเป็นบริการอิสระ deploy แยกกัน
- **ข้อดี:** scalability, independent deploy
- **ข้อเสีย:** complexity สูง, distributed system
- **เหมาะกับ:** ระบบใหญ่ ทีมหลายทีม
- **รายละเอียดเพิ่ม:** เล่ม 2

4.4.6 Hexagonal / Onion / Clean (เบื้องต้น)
- แยก business logic ออกจาก infrastructure
- Dependency ชี้เข้าหา domain
- **ข้อดี:** testable, เปลี่ยนเทคโนโลยีง่าย
- **ข้อเสีย:** ซับซ้อนขึ้นในช่วงเริ่มต้น
- **เหมาะกับ:** ระบบที่มี business logic ซับซ้อน
- **รายละเอียดเพิ่ม:** เล่ม 3–4

4.4.7 การเปรียบเทียบ

Style	Complexity	Scalability	เหมาะกับ
Monolith	ต่ำ	ต่ำ–กลาง	ระบบเล็ก ทีมเล็ก
Layered	ต่ำ–กลาง	กลาง	ระบบทั่วไป
Component-based	กลาง	กลาง–สูง	ระบบที่ต้องการ modularity
Microservices	สูง	สูง	ระบบใหญ่ ทีมหลายทีม
Event-driven	สูง	สูง	ระบบ async/reactive
Clean/Hexagonal	กลาง–สูง	กลาง	ระบบ business logic ซับซ้อน

4.5 ข้อดีของบท
- เป็นแผนที่นำทางก่อนลงลึกในเล่มถัดไป
- ทำให้เห็นทางเลือกและ trade-offs
- ช่วยให้ทีมเลือกแนวทางได้อย่างมีเหตุผล

4.6 ข้อเสีย/ข้อจำกัด
- แต่ละรูปแบบมีรายละเอียดมาก จึงเป็นเพียงภาพรวม
- ตัวอย่างอาจไม่ครอบคลุมทุกบริบท

4.7 ข้อควรระวัง
- อย่าเลือก pattern ตามกระแสโดยไม่ดู drivers
- อย่าคิดว่า style หนึ่งดีกว่าอีก style หนึ่งโดย absolute
- อย่าผสม style หลายแบบโดยไม่เข้าใจผลกระทบ

4.8 ตัวอย่าง
ทีม startup ระยะเริ่มต้นเลือก monolith เพื่อ speed และ simplicity เมื่อธุรกิจโตและทีมขยาย จึงค่อย ๆ แยกเป็น component

4.9 การประยุกต์ใช้
- ใช้ใน workshop เลือกแนวทางสถาปัตยกรรม
- ใช้เป็น checklist ว่าได้พิจารณาทางเลือกครบหรือไม่
- ใช้เชื่อมไปยังเล่ม 2–4 สำหรับรายละเอียด

4.10 สรุป/ผลลัพธ์
ผู้อ่านมีกรอบเปรียบเทียบ style/pattern และเลือกเบื้องต้นพร้อมเหตุผลได้

4.11 เชื่อมโยงกับเล่มอื่น
- เล่ม 2: Microservices
- เล่ม 3: DDD และ Hexagonal/Onion
- เล่ม 4: Clean Architecture

บทที่ 5: Architectural Drivers: Building Blocks for Decision Making

5.1 เรื่อง
ตัวขับเคลื่อนการตัดสินใจทางสถาปัตยกรรม และวิธีจัดลำดับความสำคัญ

5.2 วัตถุประสงค์
- ทำให้การตัดสินใจอิงความต้องการธุรกิจและคุณภาพระบบ
- สอนวิธีระบุและจัดลำดับ drivers
- เชื่อม drivers กับ decision record

5.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ระบุ drivers ของระบบได้
- จัดลำดับความสำคัญของ drivers ได้
- ใช้ drivers เป็นเกณฑ์ตัดสินใจสถาปัตยกรรม

5.4 หัวข้อหลัก

5.4.1 Functional Requirements vs Quality Attributes
- **Functional requirements** – ระบบทำอะไร
- **Quality attributes** – ระบบทำได้ดีแค่ไหน

สถาปัตยกรรมมักถูกขับเคลื่อนด้วย quality attributes มากกว่า functional requirements เพราะ functional requiremer

5.4.2 Business Drivers
ปัจจัยจากธุรกิจ:
- time-to-market
- ต้นทุน
- ความพึงพอใจลูกค้า
- การเติบโตทางธุรกิจ
- กฎระเบียบ

5.4.3 Technical Drivers

ปัจจัยทางเทคนิค:

- ระบบเดิมที่ต้องเชื่อมต่อ
- technology stack ขององค์กร
- ทักษะทีม
- infrastructure ที่มี
- security requirements

5.4.4 การจัดลำดับความสำคัญ

ไม่ใช่ทุก driver สำคัญเท่ากัน วิธีจัดลำดับ:

1. ระบุ drivers ทั้งหมด
2. ให้คะแนนตามผลกระทบต่อธุรกิจ
3. ให้คะแนนตามความเร่งด่วน
4. เลือก top 3–5 ที่จะใช้เป็นเกณฑ์ตัดสินใจ

5.4.5 การเชื่อม drivers กับ decision record

ทุกการตัดสินใจสำคัญควรอ้างอิง drivers:

- ตัดสินใจอะไร
- อ้างอิง driver ใด
- trade-off ที่ยอมรับ
- ผลกระทบที่คาดการณ์

5.5 ข้อดีของบท

- ทำให้การเลือกสถาปัตยกรรมตรวจสอบย้อนกลับได้
- ลดการตัดสินใจตามอารมณ์หรือกระแส
- เชื่อมสถาปัตยกรรมกับธุรกิจอย่างชัดเจน

5.6 ข้อเสีย/ข้อจำกัด

- ต้องมีผู้มีส่วนได้ส่วนเสียร่วมให้ข้อมูล
- การจัดลำดับอาจเป็นเรื่องการเมืองในองค์กร

5.7 ข้อควรระวัง

- ทุกอย่างเป็น priority สูงไม่ได้
- อย่าละเลย driver ที่ดูเหมือนเล็กน้อยแต่ส่งผลระยะยาว
- อย่าตัดสินใจโดยไม่มี driver รองรับ

5.8 ตัวอย่าง

ระบบ healthcare:

- **Drivers:** security/compliance (สูงสุด), availability (สูง), time-to-market (กลาง), scalability (กลา
- **การตัดสินใจ:** เลือก monolith ที่มี modularity ชัดเจนก่อน เพื่อลด complexity และเน้น security
- **Trade-off:** ยอมรับว่า scaling ต้องทำภายหลัง

5.9 การประยุกต์ใช้

- ทำ Architecture Driver List ก่อนเลือก style
- ใช้ใน Architecture Context Canvas (ภาคผนวก A)
- ใช้เป็นเกณฑ์ใน Style/Pattern Decision Record (ภาคผนวก B)

5.10 สรุป/ผลลัพธ์
ผู้อ่านจัดลำดับ drivers ได้ และใช้เป็นเกณฑ์ตัดสินใจสถาปัตยกรรม

5.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 6–10 (การเลือก style)
- ไปสู่เล่ม 5 (ความเสี่ยงจาก drivers ที่ไม่ชัด)
- ไปสู่เล่ม 7 (การบริหารโครงการที่อิง drivers)

ส่วนที่ 2: รูปแบบสถาปัตยกรรมและการจัดองค์ประกอบ

บทที่ 6: Monolithic Architecture

6.1 เรื่อง
สถาปัตยกรรมแบบ monolith — จุดแข็ง จุดอ่อน และบริบทที่เหมาะสม

6.2 วัตถุประสงค์
- เข้าใจโครงสร้างของ monolith
- เห็นข้อดี/ข้อเสียอย่างเป็นกลาง
- ประเมินได้ว่าเมื่อใดควรใช้หรือไม่ใช้

6.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- อธิบายโครงสร้าง monolith ได้
- ประเมินความเหมาะสมของ monolith กับบริบทได้
- รู้ว่า modular monolith คือทางเลือกที่ดีกว่าในบางกรณี

6.4 หัวข้อหลัก

6.4.1 โครงสร้างโดยทั่วไป
- โค้ดทั้งหมดอยู่ใน codebase เดียว
- build เป็น artifact เดียว
- deploy เป็น unit เดียว
- ใช้ฐานข้อมูลเดียว (มัก)

6.4.2 ข้อดี
- **เริ่มต้นง่าย** — ไม่ต้องจัดการ distributed system
- **Deploy ง่าย** — มี artifact เดียว
- **Debug ง่าย** — stack trace ต่อเนื่อง
- **Performance** — การเรียกฟังก์ชันใน process เดียวเร็ว
- **ทีมเล็ก** — ไม่ต้องแบ่งทีมตามบริการ
- **Testing** — integration test ง่ายกว่า

6.4.3 ข้อเสีย

- **Coupling** – เมื่อโตขึ้น ส่วนต่าง ๆ ผูกกันแน่น
- **Scaling** – ต้อง scale ทั้งก้อน
- **Deploy** – การเปลี่ยนแปลงเล็กน้อยต้อง deploy ทั้งระบบ
- **Technology lock-in** – ทั้งระบบใช้ stack เดียว
- **Team scaling** – ทีมใหญ่ทำงานบน codebase เดียวได้ยาก

6.4.4 Modular Monolith
ทางเลือกกลาง:
- ยังเป็น monolith แต่แบ่ง module ชัดเจน
- module สื่อสารผ่าน interface
- เตรียมพร้อมสำหรับการแยกในอนาคต

6.4.5 เมื่อใดควรใช้
- ระบบขนาดเล็ก–กลาง
- ทีมเล็ก
- ช่วงเริ่มต้นที่ยังไม่รู้ requirements ชัด
- ระบบที่ไม่ต้องการ scaling สูง
- ระบบที่ business logic ซับซ้อนแต่ไม่ต้องการ distributed complexity

6.5 ข้อดีของบท
- ลดอคติว่า monolith เป็นสิ่งเลวร้ายเสมอ
- ทำให้เห็นทางเลือก modular monolith
- ช่วยประเมินทางเลือกก่อนแยกบริการ

6.6 ข้อเสีย/ข้อจำกัด
- ไม่ได้ลงลึกถึง microservices
- ตัวอย่างอาจไม่ครอบคลุมระบบใหญ่

6.7 ข้อควรระวัง
- monolith ที่ไม่มี modularity จะแก้ยากเมื่อโต
- อย่าเลือก monolith เพราะกลัวการเปลี่ยนแปลง
- อย่าแยก microservices โดยยังไม่มีปัญหา monolith จริง

6.8 ตัวอย่าง
ระบบ ERP ขนาดเล็กสำหรับบริษัท 50 คน:
- ทีม 3 คน
- ผู้ใช้ 100 คน
- ต้องการ deploy เดือนละครั้ง
- **การตัดสินใจ:** ใช้ modular monolith – เริ่มง่าย และยังแบ่ง module ชัดเจน

6.9 การประยุกต์ใช้
- ใช้ประเมินทางเลือกก่อนแยกบริการ
- ใช้เป็นจุดเริ่มต้นสำหรับระบบใหม่
- ใช้เป็น baseline เปรียบเทียบกับ microservices

6.10 สรุป/ผลลัพธ์
ผู้อ่านประเมินความเหมาะสมของ monolith ได้ และเข้าใจว่า modular monolith เป็นทางเลือกที่สมเหตุสมผลในหลายบริบท

6.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 7 (Component-based)
- ไปสู่เล่ม 2 (Microservices – เปรียบเทียบ)

บทที่ 7: Component Based Architecture

7.1 เรื่อง
การออกแบบสถาปัตยกรรมด้วย component – boundary, contract และการสื่อสาร

7.2 วัตถุประสงค์
- เข้าใจว่า component คืออะไรและต่างจาก class อย่างไร
- ออกแบบ boundary และ contract ได้
- เห็นความเชื่อมโยงกับ modularity และ microservices

7.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- กำหนด component และ boundary ได้
- วาด component diagram เบื้องต้นได้
- ออกแบบ contract ระหว่าง component ได้

7.4 หัวข้อหลัก

7.4.1 Component คืออะไร
Component คือหน่วยสถาปัตยกรรมที่มี:
- **ความรับผิดชอบชัดเจน** – ทำอะไรอย่างเดียว
- **Boundary** – มีขอบเขตที่ชัดเจน
- **Contract** – สื่อสารผ่าน interface ที่กำหนด
- **Independent deployability** (ในบางบริบท)

Component ไม่ใช่ class แต่เป็นกลุ่มของ class/module ที่ทำงานร่วมกัน

7.4.2 Boundary และ Contract
- **Boundary** – ขอบเขตที่บอกว่าอะไรอยู่ข้างใน อะไรอยู่ข้างนอก
- **Contract** – ข้อตกลงการสื่อสาร เช่น API, interface, message format
- Contract ที่ดีควร stable และไม่ผูกกับ implementation

7.4.3 Components vs Classes
- Class = หน่วยโค้ด
- Component = หน่วยสถาปัตยกรรม
- Component หนึ่งประกอบด้วยหลาย class
- การออกแบบ component ต้องคิดถึงความรับผิดชอบและ boundary ไม่ใช่แค่ class diagram

7.4.4 การสื่อสารระหว่าง Component
- **Synchronous** – เรียกใช้โดยตรง (function call, HTTP)

- **Asynchronous** – ผ่าน event/message queue
- **Shared data** – ผ่านฐานข้อมูล (ระวัง coupling)

7.4.5 การกำหนดความรับผิดชอบ

หลักการ:
- **Single Responsibility** – component หนึ่งมีความรับผิดชอบหลักหนึ่ง
- **High Cohesion** – สิ่งที่เกี่ยวข้องกันอยู่ด้วยกัน
- **Low Coupling** – พึ่งพาน้อยที่สุด

7.5 ข้อดีของบท
- เป็นฐานให้เข้าใจ microservices และ modularity
- ทำให้เห็นความสำคัญของ boundary
- ใช้ได้ทั้ง monolith และ distributed system

7.6 ข้อเสีย/ข้อจำกัด
- การแบ่ง component ต้องอาศัยบริบทโดเมน
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

7.7 ข้อควรระวัง
- component ใหญ่เกินไป (god component)
- component เล็กเกินไป (over-fragmentation)
- boundary ที่ไม่ชัดทำให้ coupling แอบแฝง

7.8 ตัวอย่าง

ระบบ e-commerce แบ่งเป็น component:
- **Catalog** – จัดการสินค้า
- **Order** – จัดการคำสั่งซื้อ
- **Payment** – จัดการชำระเงิน
- **Notification** – แจ้งเตือน

แต่ละ component มี contract ชัดเจน Order เรียก Payment ผ่าน interface ไม่ผูกกับ implementation

7.9 การประยุกต์ใช้
- ใช้ทำ component map ของระบบ
- ใช้กำหนด boundary ก่อน refactor
- ใช้เป็นพื้นฐานสำหรับ microservices

7.10 สรุป/ผลลัพธ์

ผู้อ่านกำหนด component, boundary และ contract ได้ และเห็นความเชื่อมโยงกับ modularity

7.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 14 (Components vs Classes)
- ไปสู่บทที่ 17 (Package by Component)
- ไปสู่เล่ม 2 (Microservices)

บทที่ 8: PresentationDomainDataLayering

8.1 เรื่อง
การแบ่งชั้นแบบ Presentation–Domain–Data และ dependency ระหว่างชั้น

8.2 วัตถุประสงค์
- เข้าใจการแบ่ง layer พื้นฐาน
- เห็นข้อดี/ข้อเสียของ layered architecture
- รู้ว่า domain layer ควรมี business logic

8.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ออกแบบ layer เบื้องต้นได้
- กำหนด dependency direction ได้
- หลีกเลี่ยงปัญหา domain layer บางเกินไป

8.4 หัวข้อหลัก

8.4.1 แต่ละชั้นรับผิดชอบอะไร
- **Presentation** – UI, API controller, การรับส่งข้อมูล
- **Domain** – business logic, rules, validation
- **Data** – การเข้าถึงฐานข้อมูล, repository

8.4.2 Dependency ระหว่างชั้น
- Presentation → Domain → Data
- Domain ไม่ควรรู้จัก Presentation หรือ Data โดยตรง
- Data implement interface ที่ Domain กำหนด (Dependency Inversion)

8.4.3 ข้อดี
- แยกหน้าที่ชัดเจน
- ทดสอบง่ายขึ้น
- เปลี่ยน UI หรือ database ได้โดยกระทบน้อย

8.4.4 ข้อเสีย
- อาจกลายเป็น "lasagna code" ที่ layer เยอะเกิน
- Domain layer อาจบางเกินไปถ้าไม่ระวัง
- การเปลี่ยนแปลงอาจต้องแก้หลาย layer

8.4.5 Domain Layer ที่มี Business Logic
ปัญหาที่พบบ่อย: domain layer กลายเป็นเพียงที่ส่งต่อข้อมูลระหว่าง presentation กับ data โดยไม่มี logic จริง

วิธีแก้:
- ย้าย business rules จาก controller/service ไป domain
- ใช้ domain model ที่มี behavior ไม่ใช่แค่ data
- ทดสอบ domain layer โดยตรง

8.4.6 การทดสอบแต่ละชั้น

- Presentation – UI test, API test
- Domain – unit test (สำคัญที่สุด)
- Data – integration test

8.5 ข้อดีของบท
- เป็นพื้นฐานของ layered architecture
- ทำให้เห็นความสำคัญของ domain layer
- ใช้ได้กับระบบขนาดเล็ก–กลาง

8.6 ข้อเสีย/ข้อจำกัด
- อาจไม่พอสำหรับโดเมนซับซ้อน
- ไม่ได้ลงลึกถึง DDD

8.7 ข้อควรระวัง
- domain กลายเป็นชั้นบาง ๆ ที่ไม่มี logic
- layer เยอะเกินจน维护ยาก
- dependency ผิดทิศ

8.8 ตัวอย่าง
Web app สำหรับจัดการสินค้า:
- **Presentation:** REST controller รับ request
- **Domain:** Product entity, PriceCalculator, StockValidator
- **Data:** ProductRepository ที่ implement interface จาก Domain

8.9 การประยุกต์ใช้
- ใช้ตรวจโครงสร้างโปรเจกต์ขนาดเล็ก
- ใช้เป็นจุดเริ่มต้นก่อนปรับเป็น Clean Architecture

8.10 สรุป/ผลลัพธ์
ผู้อ่านแยกหน้าที่ของ layer ได้ และเข้าใจว่า domain layer ต้องมี business logic จริง

8.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 9–10
- ไปสู่เล่ม 4 (Clean Architecture)

บทที่ 9: Layers, Hexagons, Features and Components

9.1 เรื่อง
ทางเลือกในการจัดโครงสร้างนอกเหนือจาก layer – hexagonal, feature-based, component-based

9.2 วัตถุประสงค์
- เปรียบเทียบแนวทางจัดองค์ประกอบหลายแบบ
- ทำให้เห็นว่าการแบ่งตาม layer ไม่ใช่ทางเดียว
- ช่วยเลือกแนวทางให้เหมาะกับบริบท

9.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- อธิบายความแตกต่างของแต่ละแนวทางได้
- เลือกแนวทางจัดโครงสร้างให้เหมาะกับทีมและระบบ
- รู้ว่าแต่ละแนวทางเชื่อมกับเล่มใด

9.4 หัวข้อหลัก

9.4.1 Layer-Based
- แบ่งตามชั้น (presentation, domain, data)
- **ข้อดี:** เข้าใจง่าย คุ้นเคย
- **ข้อเสีย:** อาจซ่อน domain, การเปลี่ยนแปลง feature ต้องแก้หลาย layer

9.4.2 Hexagonal (Ports & Adapters)
- แยก core domain ออกจาก infrastructure ผ่าน port/adapter
- **ข้อดี:** testable, เปลี่ยนเทคโนโลยีง่าย
- **ข้อเสีย:** ต้องออกแบบ port ให้ดี
- **รายละเอียดเพิ่ม:** เล่ม 3

9.4.3 Feature-Based
- แบ่งตาม feature (เช่น order, catalog, payment)
- **ข้อดี:** เปลี่ยนแปลง feature ได้ในที่เดียว
- **ข้อเสีย:** อาจมี duplication ข้าม feature

9.4.4 Component-Based
- แบ่งตาม component ที่มี boundary ชัด
- **ข้อดี:** modularity สูง
- **ข้อเสีย:** ต้องออกแบบ boundary ดี

9.4.5 เปรียบเทียบ

แนวทาง	ข้อดี	ข้อเสีย	เหมาะกับ
Layer	ง่าย คุ้นเคย	ซ่อน domain	ระบบเล็ก
Hexagonal	testable, flexible	ซับซ้อนขึ้น	ระบบ business logic ซับซ้อน
Feature	เปลี่ยนแปลงง่าย	duplication	ระบบที่ feature ชัด
Component	modularity	ต้องออกแบบ boundary	ระบบที่ต้องการแยกส่วน

9.5 ข้อดีของบท
- ทำให้เห็นทางเลือกมากกว่าการแบ่งตาม layer
- ช่วยให้ทีมเลือกแนวทางได้อย่างมีเหตุผล

9.6 ข้อเสีย/ข้อจำกัด
- เป็นภาพรวม ไม่ใช่คู่มือ implementation
- แต่ละแนวทางมีรายละเอียดมาก

9.7 ข้อควรระวัง

- การจัดตาม layer อาจซ่อน domain
- อย่าเลือกแนวทางตามกระแส
- อย่าผสมหลายแนวทางโดยไม่เข้าใจ

9.8 ตัวอย่าง
ระบบเดียวกันจัดโครงสร้าง 3 แบบ:
- **Layer:** `presentation/`, `domain/`, `data/`
- **Feature:** `order/`, `catalog/`, `payment/`
- **Component:** `order-component/`, `catalog-component/`

แต่ละแบบมีผลต่อการ maintain และการทดสอบต่างกัน

9.9 การประยุกต์ใช้
- ใช้เลือกโครงสร้างโปรเจกต์ใหม่
- ใช้ทบทวนโครงสร้างเดิม
- ใช้ใน workshop ออกแบบ package

9.10 สรุป/ผลลัพธ์
ผู้อ่านเลือกแนวทางจัดโครงสร้างได้อย่างมีเหตุผล

9.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 10, 18
- ไปสู่เล่ม 3–4

บทที่ 10: More than Concentric Layers

10.1 เรื่อง
ข้อจำกัดของ concentric layers และสะพานเชื่อมไปสู่ DDD และ Clean Architecture

10.2 วัตถุประสงค์
- ทำให้เห็นว่าทำไม layer แบบ concentric อาจไม่พอ
- เชื่อมแนวคิดไปสู่การออกแบบที่อิงโดเมน
- เตรียมผู้อ่านก่อนเข้าเล่ม 3–4

10.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- อธิบายข้อจำกัดของ concentric layers ได้
- เข้าใจว่าทำไมต้องสะท้อนโดเมนในโครงสร้าง
- เห็นเส้นทางไปสู่ DDD และ Clean Architecture

10.4 หัวข้อหลัก

10.4.1 ข้อจำกัดของ Concentric Layers
- Layer แบบดั้งเดิมอาจทำให้ domain กลายเป็นชั้นบาง ๆ
- การเปลี่ยนแปลง feature ต้องแก้หลาย layer

- ไม่ได้สะท้อน bounded context หรือ domain จริง

10.4.2 ทำไมต้องสะท้อนโดเมน
- โดเมนคือหัวใจของธุรกิจ
- โครงสร้างที่สะท้อนโดเมนทำให้สื่อสารง่าย
- ลด coupling ระหว่าง domain ที่ต่างกัน
- เตรียมพร้อมสำหรับการแยกบริการ

10.4.3 บทนำไปสู่ DDD/Clean Architecture
- **DDD** – แยก bounded context, ใช้ ubiquitous language, ออกแบบ aggregate
- **Clean Architecture** – แยกชั้นตาม dependency rule, ให้ domain อยู่ศูนย์กลาง
- ทั้งสองแนวทางทำให้สถาปัตยกรรมสะท้อนโดเมนมากกว่าเทคโนโลยี

10.4.4 Dependency Rule เบื้องต้น
- Dependency ชี้เข้าหา domain
- Domain ไม่รู้จัก infrastructure
- Infrastructure implement interface ที่ domain กำหนด

10.5 ข้อดีของบท
- เป็นบทเชื่อมสำคัญไปเล่ม 3–4
- ทำให้ผู้อ่านเห็นว่าทำไมต้องไปต่อ
- ลดการติดอยู่กับ layered architecture แบบดั้งเดิม

10.6 ข้อเสีย/ข้อจำกัด
- ยังไม่ลงลึกเรื่อง DDD
- ต้องอ่านเล่ม 3–4 เพื่อความเข้าใจเต็ม

10.7 ข้อควรระวัง
- อย่าแยกชั้นจน business logic กระจัดกระจาย
- อย่าคิดว่า Clean Architecture คือคำตอบทุกปัญหา
- อย่าละเลยความซับซ้อนที่เพิ่มขึ้น

10.8 ตัวอย่าง
ระบบที่มี business rules ซับซ้อน (เช่น การคำนวณภาษีหลายรูปแบบ) – layered architecture แบบดั้งเดิมทำให้ logic กร

10.9 การประยุกต์ใช้
- ใช้ตั้งคำถามก่อน refactor สถาปัตยกรรม
- ใช้เป็นจุดตัดสินใจว่าจะไป DDD/Clean หรือไม่

10.10 สรุป/ผลลัพธ์
ผู้อ่านเห็นเส้นทางไปสู่สถาปัตยกรรมที่อิงโดเมน และเข้าใจข้อจำกัดของ concentric layers

10.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่เล่ม 3 (DDD)
- ไปสู่เล่ม 4 (Clean Architecture)

ส่วนที่ 3: จากสถาปัตยกรรมสู่โค้ด

บทที่ 11: Software Architecture vs Code

11.1 เรื่อง
ความสัมพันธ์ระหว่างสถาปัตยกรรมกับโค้ด และปัญหา drift ระหว่างกัน

11.2 วัตถุประสงค์
- ทำให้เห็นว่าสถาปัตยกรรมปรากฏในโค้ดอย่างไร
- อธิบายปัญหา drift และวิธีตรวจสอบ
- เตรียมพื้นฐานสำหรับบทที่ 12–13

11.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ตรวจสอบว่าโค้ดสะท้อนสถาปัตยกรรมหรือไม่
- ระบุสัญญาณเตือนของ drift ได้
- วางแนวทางป้องกัน drift

11.4 หัวข้อหลัก

11.4.1 สถาปัตยกรรมปรากฏในโค้ดอย่างไร
- โครงสร้าง package/module
- dependency ระหว่าง package
- interface/contract
- ชื่อและขอบเขตของ component

11.4.2 Drift ระหว่าง Design กับ Code
Drift คือภาวะที่เอกสารสถาปัตยกรรมกับโค้ดจริงไม่ตรงกัน เกิดจาก:
- การเปลี่ยนแปลงโค้ดโดยไม่อัปเดตเอกสาร
- การตัดสินใจที่ไม่ถูกบันทึก
- การขาดการตรวจสอบ

11.4.3 วิธีตรวจสอบ
- Architecture review
- Static analysis (dependency check)
- Code review ที่เน้น boundary
- การเทียบ package structure กับ component diagram

11.4.4 สัญญาณเตือน
- package ที่พึ่งพากันแบบ cyclic
- component ที่ใหญ่เกินไป
- การเรียกข้าม layer โดยตรง
- ชื่อ package ที่ไม่สื่อความหมาย

11.5 ข้อดีของบท
- เชื่อมเอกสารกับงานจริง
- ทำให้เห็นปัญหา drift ก่อนลุกลาม
- เตรียมเครื่องมือสำหรับบทถัดไป

11.6 ข้อเสีย/ข้อจำกัด
- ต้องอาศัยตัวอย่างโค้ดประกอบ
- การตรวจสอบอัตโนมัติต้องมีเครื่องมือ

11.7 ข้อควรระวัง
- เอกสารกับโค้ดไม่ตรงกัน
- ปล่อย drift สะสมจนแก้ยาก
- คิดว่าการมี diagram เท่ากับมีสถาปัตยกรรม

11.8 ตัวอย่าง
Component diagram ระบุว่า Order ไม่ควรเรียก Data โดยตรง แต่ในโค้ดพบว่า OrderController เรียก Database โดยตรง –

11.9 การประยุกต์ใช้
- ใช้ใน architecture review
- ใช้ตั้ง checklist สำหรับ code review
- ใช้เป็น baseline ก่อน refactor

11.10 สรุป/ผลลัพธ์
ผู้อ่านตรวจ alignment ระหว่างสถาปัตยกรรมกับโค้ดได้ และรู้จักสัญญาณเตือนของ drift

11.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 12–13, 19
- ไปสู่เล่ม 4 (การรักษา dependency rule)

บทที่ 12: Mapping Software Architecture to Code

12.1 เรื่อง
การแปลงสถาปัตยกรรมเป็นโครงสร้างโค้ด – component สู่ package/module

12.2 วัตถุประสงค์
- สอนวิธี map component สู่ package
- อธิบายการกำหนด interface และ dependency direction
- ทำให้สถาปัตยกรรมนำไปใช้ได้จริง

12.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- สร้าง architecture-to-code map ได้
- กำหนด package structure ที่สะท้อนสถาปัตยกรรม
- กำหนด dependency direction ที่ถูกต้อง

12.4 หัวข้อหลัก

12.4.1 Component → Package/Module
- แต่ละ component ควรมี package ของตนเอง
- ชื่อ package ควรสื่อ component
- หลีกเลี่ยง package ที่รวมหลาย component

12.4.2 Interface/Contract
- กำหนด interface ที่ boundary ของ component
- interface ควร stable และไม่ผูกกับ implementation
- ใช้ dependency inversion เมื่อเหมาะสม

12.4.3 Dependency Direction
- Dependency ควรชี้ในทิศทางที่กำหนด
- หลีกเลี่ยง cyclic dependency
- ใช้เครื่องมือตรวจสอบ

12.4.4 การตั้งชื่อ
- ชื่อ package ควรสื่อความหมายทางธุรกิจ
- หลีกเลี่ยงชื่อตามเทคโนโลยี (utils, helpers)
- ใช้ ubiquitous language ถ้ามี

12.4.5 ตัวอย่าง Mapping
Component diagram:
- Order → Payment (ผ่าน PaymentService interface)

Package structure:

order/ OrderService.java PaymentService.java (interface) payment/ PaymentServiceImpl.java

12.5 ข้อดีของบท
- ทำให้สถาปัตยกรรมนำไปใช้ได้จริง
- ลดช่องว่างระหว่าง design กับ code
- เป็นพื้นฐานของ package alignment

12.6 ข้อเสีย/ข้อจำกัด
- mapping 1:1 อาจไม่เหมาะทุกกรณี
- ต้องปรับตามภาษาและ framework

12.7 ข้อควรระวัง
- mapping ที่ซับซ้อนเกินจำเป็น
- package ที่ใหญ่เกินไปหรือเล็กเกินไป
- ละเลย dependency direction

12.8 ตัวอย่าง

ระบบ e-commerce:
- Component: Catalog, Order, Payment
- Package: `catalog/`, `order/`, `payment/`
- Order เรียก Payment ผ่าน `PaymentService` interface
- Payment implement interface ใน package ของตนเอง

12.9 การประยุกต์ใช้
- ใช้ตั้งโครงโปรเจกต์ใหม่
- ใช้ refactor โปรเจกต์เดิม
- ใช้ตรวจ alignment

12.10 สรุป/ผลลัพธ์
ผู้อ่านสร้าง architecture-to-code map ได้ และกำหนด package structure ที่สะท้อนสถาปัตยกรรม

12.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 13, 17–18
- ไปสู่เล่ม 4 (Clean Architecture)

บทที่ 13: An Architecturally-Evident Coding Style

13.1 เรื่อง
สไตล์โค้ดที่ทำให้เห็นสถาปัตยกรรมได้ด้วยตัวเอง

13.2 วัตถุประสงค์
- กำหนดแนวทางเขียนโค้ดที่สะท้อนสถาปัตยกรรม
- ทำให้โค้ดสื่อสารได้โดยไม่ต้องอ่านเอกสาร
- เตรียมพื้นฐานสำหรับการตรวจสอบ alignment

13.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ตั้ง coding style ที่สะท้อนสถาปัตยกรรม
- ระบุสัญญาณเตือนเมื่อโค้ดเบี่ยงจากสถาปัตยกรรม
- ใช้ style เป็นเครื่องมือสื่อสารในทีม

13.4 หัวข้อหลัก

13.4.1 ชื่อ Package สื่อความหมาย
- ใช้ชื่อที่สื่อ component/domain
- หลีกเลี่ยงชื่อกว้าง ๆ เช่น `common`, `utils`
- ใช้ ubiquitous language ถ้ามี

13.4.2 การจัดกลุ่มไฟล์
- จัดกลุ่มตาม component/feature
- หลีกเลี่ยงการจัดกลุ่มตามเทคโนโลยีเพียงอย่างเดียว
- ให้โครงสร้างสื่อความรับผิดชอบ

```
#### 13.4.3 Dependency ที่มองเห็นได้
- import ที่อ่านแล้วเห็นทิศทาง
- หลีกเลี่ยงการเรียกข้าม layer โดยตรง
- ใช้ interface ที่ boundary

#### 13.4.4 สัญญาณเตือนเมื่อโค้ดเบี่ยงจากสถาปัตยกรรม
- package ที่ import กันแบบ cyclic
- component ที่เรียก database โดยตรง
- ชื่อ class ที่ไม่สื่อ domain
- ไฟล์ที่ใหญ่มากใน package เดียว

### 13.5 ข้อดีของบท
- ทำให้โค้ดสื่อสารสถาปัตยกรรมได้ด้วยตัวเอง
- ลดการพึ่งพาเอกสาร
- ช่วยให้ code review เห็นปัญหาได้เร็ว

### 13.6 ข้อเสีย/ข้อจำกัด
- ต้องมีวินัยและการレビュー
- อาจต้องปรับตามภาษา/framework

### 13.7 ข้อควรระวัง
- ชื่อสวยแต่โครงสร้างไม่จริง
- style ที่เข้มเกินไปอาจทำให้ทีมต่อต้าน
- ละเลยการอัปเดต style เมื่อสถาปัตยกรรมเปลี่ยน

### 13.8 ตัวอย่าง
โครงสร้างที่สะท้อนสถาปัตยกรรม:
```

order/ domain/ Order.java OrderRepository.java (interface) application/ OrderService.java infrastructure/ JpaOrderRepository.java api/ OrderController.java

```
อ่านแล้วเห็น layer และ dependency direction

### 13.9 การประยุกต์ใช้
- ใช้เป็น guideline ของทีม
- ใช้ใน code review
- ใช้ตั้ง template โปรเจกต์ใหม่

### 13.10 สรุป/ผลลัพธ์
ผู้อ่านกำหนด coding style ที่สะท้อนสถาปัตยกรรมได้ และใช้เป็นเครื่องมือสื่อสาร

### 13.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 14–15, 19
- ไปสู่เล่ม 4 (โครงสร้างที่สะท้อน Clean Architecture)
```

```
---

## บทที่ 14: Components vs Classes

### 14.1 เรื่อง
ความแตกต่างระหว่างหน่วยโค้ด (class) และหน่วยสถาปัตยกรรม (component)

### 14.2 วัตถุประสงค์
- แยกบทบาทของ class และ component
- ทำให้การออกแบบไม่สับสนระหว่างสองระดับ
- เตรียมพื้นฐานสำหรับ package by component

### 14.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- แยกหน่วยโค้ดกับหน่วยสถาปัตยกรรมได้
- ออกแบบ component โดยไม่สับสนกับ class
- กำหนด boundary ของ component ได้

### 14.4 หัวข้อหลัก

#### 14.4.1 Class คือหน่วยโค้ด
- หน่วยที่เล็กที่สุดของการออกแบบ OO
- มี state และ behavior
- อยู่ใน component

#### 14.4.2 Component คือหน่วยสถาปัตยกรรม
- กลุ่มของ class ที่ทำงานร่วมกัน
- มี boundary และ contract
- สื่อสารกับ component อื่นผ่าน interface

#### 14.4.3 ความสัมพันธ์และการแบ่งหน้าที่
- Component หนึ่งมีหลาย class
- Class ไม่ควรข้าม boundary โดยตรง
- การออกแบบ component คิดที่ระดับความรับผิดชอบ ไม่ใช่ class diagram

#### 14.4.4 Boundary ของ Component
- กำหนดจากความรับผิดชอบทางธุรกิจ
- ไม่ควรผูกกับ technology stack
- ควรทดสอบได้แยกจากกัน

### 14.5 ข้อดีของบท
- ลดความสับสนในการออกแบบ
- ทำให้เห็นความสำคัญของ boundary
- เชื่อมโยงกับ modularity

### 14.6 ข้อเสีย/ข้อจำกัด
- ต้องมีตัวอย่างประกอบ
```

- ต้องอาศัยความเข้าใจโดเมน

14.7 ข้อควรระวัง
- ใช้ class แทน component จน boundary หาย
- component ที่มี class เดียว (อาจเล็กเกินไป)
- ละเลย contract ระหว่าง component

14.8 ตัวอย่าง
Component `Order` ประกอบด้วย:
- `Order` (entity)
- `OrderItem` (entity)
- `OrderService` (application service)
- `OrderRepository` (interface)
- `JpaOrderRepository` (implementation)

ทั้งหมดนี้คือ component เดียว ไม่ใช่หลาย component

14.9 การประยุกต์ใช้
- ใช้ตรวจ component map
- ใช้สอนทีมเรื่องระดับการออกแบบ
- ใช้ใน code review

14.10 สรุป/ผลลัพธ์
ผู้อ่านแยกหน่วยโค้ดกับหน่วยสถาปัตยกรรมได้ และออกแบบ component ได้ถูกระดับ

14.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 7, 17
- ไปสู่เล่ม 2 (บริการใน microservices)

บทที่ 15: Reflecting Architecture and Domain in Code

15.1 เรื่อง
การทำให้โค้ดสะท้อนทั้งสถาปัตยกรรมและโดเมน

15.2 วัตถุประสงค์
- รวมแนวคิด architecture และ domain ในโครงสร้างโค้ด
- ทำให้โค้ดสื่อความหมายทางธุรกิจ
- เตรียมพื้นฐานสำหรับ DDD

15.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ออกแบบโครงสร้างโค้ดที่สื่อทั้ง architecture และ domain
- หลีกเลี่ยงโครงสร้างตาม framework
- รักษา dependency rule

```
### 15.4 หัวข้อหลัก

#### 15.4.1 โครงสร้างที่สื่อ Domain
- ใช้ชื่อตาม ubiquitous language
- จัดกลุ่มตาม bounded context (ถ้ามี)
- หลีกเลี่ยงชื่อตามเทคโนโลยี

#### 15.4.2 โครงสร้างที่สื่อ Architecture
- แยก layer/component ตามสถาปัตยกรรม
- dependency ชี้ในทิศทางที่กำหนด
- boundary ชัดเจน

#### 15.4.3 การหลีกเลี่ยงโครงสร้างตาม Framework
- อย่าให้ framework กำหนดโครงสร้างหลัก
- ใช้ framework เป็น infrastructure
- domain ไม่ควรขึ้นกับ framework

#### 15.4.4 การรักษา Dependency Rule
- domain ไม่รู้จัก infrastructure
- infrastructure implement interface จาก domain
- ใช้ dependency injection

### 15.5 ข้อดีของบท
- เป็นสะพานไป DDD และ Clean Architecture
- ทำให้โค้ดสื่อความหมายทั้งธุรกิจและสถาปัตยกรรม
- ลด coupling กับ framework

### 15.6 ข้อเสีย/ข้อจำกัด
- ต้องเข้าใจโดเมนพอสมควร
- อาจต้องปรับโครงสร้างเดิมมาก

### 15.7 ข้อควรระวัง
- พยายามสะท้อนทุกอย่างจนซับซ้อนเกิน
- ละเลยการทดสอบ
- ใช้โครงสร้างตาม framework โดยไม่รู้ตัว

### 15.8 ตัวอย่าง
โครงสร้างที่สะท้อนทั้ง domain และ architecture:
```

sales/ ← bounded context domain/ ← **ไม่ขึ้นกับ** infrastructure Order.java OrderRepository.java application/ PlaceOrderUseCase.java infrastructure/ JpaOrderRepository.java api/ OrderController.java

```
### 15.9 การประยุกต์ใช้
- ใช้ปรับโครงสร้างก่อนทำ DDD
- ใช้เป็น guideline ของทีม
```

- ใช้ใน architecture review

15.10 สรุป/ผลลัพธ์
ผู้อ่านออกแบบโครงสร้างโค้ดที่สื่อทั้ง architecture และ domain ได้

15.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่เล่ม 3 (DDD)
- ไปสู่เล่ม 4 (Clean Architecture)

ส่วนที่ 4: Modularity, Packaging และการทดสอบ

บทที่ 16: Modularity and Testability

16.1 เรื่อง
ความเชื่อมโยงระหว่าง modularity กับการทดสอบ

16.2 วัตถุประสงค์
- ทำให้เห็นว่าการออกแบบที่ดีทดสอบง่ายกว่า
- อธิบาย coupling/cohesion และผลต่อการทดสอบ
- เตรียมพื้นฐานสำหรับ test strategy

16.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ประเมิน testability ของสถาปัตยกรรมได้
- ระบุปัญหา modularity ที่ทำให้ทดสอบยาก
- ออกแบบ component ที่ทดสอบง่าย

16.4 หัวข้อหลัก

16.4.1 Coupling/Cohesion
- **Coupling ต่ำ** – component พึ่งพากันน้อย
- **Cohesion สูง** – สิ่งที่เกี่ยวข้องกันอยู่ด้วยกัน
- ทั้งสองส่งผลโดยตรงต่อ testability

16.4.2 Test Pyramid เบื้องต้น
- Unit test (ฐาน) – เร็ว จำนวนมาก
- Integration test (กลาง) – ทดสอบการเชื่อมต่อ
- E2E test (ยอด) – ช้า จำนวนน้อย

สถาปัตยกรรมที่ดีทำให้ unit test ครอบคลุมได้มาก

16.4.3 การทดสอบ Component
- ทดสอบที่ boundary

- ใช้ test double สำหรับ dependency ภายนอก
- หลีกเลี่ยงการทดสอบข้าม boundary โดยไม่ตั้งใจ

16.4.4 Dependency Injection เบื้องต้น
- ช่วยให้ทดสอบได้โดยไม่ต้องสร้าง dependency จริง
- ลด coupling ระหว่าง component
- ทำให้เปลี่ยน implementation ได้ง่าย

16.5 ข้อดีของบท
- เชื่อมสถาปัตยกรรมกับคุณภาพการทดสอบ
- ทำให้เห็นว่าการออกแบบดีทดสอบง่าย
- เตรียมพื้นฐานสำหรับบทที่ 17

16.6 ข้อเสีย/ข้อจำกัด
- ไม่ได้ลงลึกเครื่องมือทดสอบ
- ตัวอย่างอาจไม่ครอบคลุมทุกภาษา

16.7 ข้อควรระวัง
- modularity ที่ไม่จริงทำให้ทดสอบยาก
- การทดสอบที่ผูกกับ implementation มากเกินไป
- ละเลย integration test

16.8 ตัวอย่าง
Component ที่ทดสอบง่าย:
- OrderService รับ OrderRepository เป็น interface
- ทดสอบด้วย in-memory repository
- ไม่ต้องใช้ฐานข้อมูลจริง

Component ที่ทดสอบยาก:
- OrderService สร้าง database connection เอง
- ต้องใช้ฐานข้อมูลจริงในการทดสอบ

16.9 การประยุกต์ใช้
- ใช้ประเมินก่อน refactor
- ใช้กำหนด test strategy
- ใช้ใน code review

16.10 สรุป/ผลลัพธ์
ผู้อ่านประเมิน testability ได้ และออกแบบ component ที่ทดสอบง่าย

16.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 17
- ไปสู่เล่ม 4 (การทดสอบ Clean Architecture)

บทที่ 17: Package by Component and Architecturally-Aligned Testing

17.1 เรื่อง
การจัด package ตาม component และการทดสอบที่สอดคล้องกับสถาปัตยกรรม

17.2 วัตถุประสงค์
- เชื่อมโครงสร้าง package กับกลยุทธ์การทดสอบ
- ทำให้การทดสอบสะท้อน boundary ของสถาปัตยกรรม
- เตรียมพื้นฐานสำหรับการรักษา alignment

17.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- ออกแบบ package ตาม component
- ออกแบบ test strategy ที่สอดคล้อง
- ป้องกันการทดสอบข้าม boundary

17.4 หัวข้อหลัก

17.4.1 Package by Component
- แต่ละ component มี package ของตนเอง
- package ภายในจัดตาม layer/role
- dependency ระหว่าง package สะท้อนสถาปัตยกรรม

17.4.2 Testing ตาม Boundary
- ทดสอบ component ที่ boundary
- ใช้ contract test สำหรับ interface
- หลีกเลี่ยงการทดสอบที่รู้จัก implementation ภายใน

17.4.3 Contract Test เบื้องต้น
- ทดสอบว่า implementation ตรงตาม contract
- ใช้เมื่อมีหลาย implementation
- ช่วยป้องกันการเปลี่ยนแปลงที่กระทบผู้ใช้ contract

17.4.4 การป้องกันการทดสอบข้าม Boundary
- จำกัด access modifier
- ใช้ package-private เมื่อเหมาะสม
- ตรวจสอบ dependency ใน test code

17.5 ข้อดีของบท
- ทำให้การทดสอบสะท้อนสถาปัตยกรรม
- ลดการทดสอบที่เปราะบาง
- เตรียมพร้อมสำหรับการเปลี่ยนแปลง

17.6 ข้อเสีย/ข้อจำกัด
- ต้องมีความรู้เรื่อง test strategy
- อาจต้องปรับโครงสร้างเดิม

17.7 ข้อควรระวัง

```
- ทดสอบข้าม boundary โดยไม่ตั้งใจ
- test ที่ผูกกับ implementation มากเกินไป
- ละเลย contract test

### 17.8 ตัวอย่าง
Package structure:
```

order/ domain/ application/ infrastructure/ api/

```
Test:
- Unit test ใน `domain/` และ `application/`
- Contract test สำหรับ `OrderRepository`
- Integration test ใน `infrastructure/`
- API test ใน `api/`

### 17.9 การประยุกต์ใช้
- ใช้ตั้งโครงทดสอบโปรเจกต์ใหม่
- ใช้ปรับ test strategy เดิม
- ใช้ใน code review

### 17.10 สรุป/ผลลัพธ์
ผู้ออกแบบ package และ test strategy ที่สอดคล้องกันได้

### 17.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 18–19
- ไปสู่เล่ม 4 (การทดสอบ Use Case)

---

## บทที่ 18: Schools of Package Architecture – An Illustration

### 18.1 เรื่อง
แนวทางจัด package หลายสำนัก และผลกระทบต่อการ maintain

### 18.2 วัตถุประสงค์
- เปรียบเทียบแนวทาง package architecture
- ทำให้เห็นข้อดี/ข้อเสียของแต่ละแบบ
- ช่วยเลือกแนวทางให้เหมาะกับบริบท

### 18.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- อธิบายแนวทาง package หลัก ๆ ได้
- เลือกแนวทาง package ได้อย่างมีเหตุผล
- เข้าใจผลกระทบต่อการ maintain และการทดสอบ

### 18.4 หัวข้อหลัก
```

```
#### 18.4.1 Package by Layer
```

presentation/ domain/ data/

```
- **ข้อดี:** คุ้นเคย, ง่าย
- **ข้อเสีย:** feature กระจาย, domain อาจบาง

#### 18.4.2 Package by Feature
```

order/ catalog/ payment/

```
- **ข้อดี:** เปลี่ยนแปลง feature ได้ในที่เดียว
- **ข้อเสีย:** อาจมี duplication ข้าม feature

#### 18.4.3 Package by Component
```

order-component/ catalog-component/ payment-component/

```
- **ข้อดี:** boundary ชัด, modularity สูง
- **ข้อเสีย:** ต้องออกแบบ boundary ดี

#### 18.4.4 Package by Domain (Bounded Context)
```

sales/ order/ customer/ inventory/ stock/ warehouse/

```
- **ข้อดี:** สะท้อนโดเมน, เหมาะกับ DDD
- **ข้อเสีย:** ต้องเข้าใจโดเมนดี

#### 18.4.5 เปรียบเทียบ

| แนวทาง | ข้อดี | ข้อเสีย | เหมาะกับ |
|---|---|---|---|
| Layer | ง่าย | feature กระจาย | ระบบเล็ก |
| Feature | เปลี่ยนแปลงง่าย | duplication | ระบบที่ feature ชัด |
| Component | boundary ชัด | ต้องออกแบบดี | ระบบที่ต้องการ modularity |
| Domain | สะท้อนธุรกิจ | ต้องเข้าใจโดเมน | ระบบที่ใช้ DDD |

### 18.5 ข้อดีของบท
- ทำให้เห็นทางเลือกและผลกระทบ
- ช่วยให้ทีมเลือกได้อย่างมีเหตุผล
- เป็นพื้นฐานของ architecture review
```

18.6 ข้อเสีย/ข้อจำกัด
- ไม่มีแบบเดียวที่เหมาะทุกทีม
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

18.7 ข้อควรระวัง
- เลือกตามกระแสโดยไม่ดูบริบท
- ผสมหลายแนวทางโดยไม่เข้าใจ
- ละเลยการทดสอบ

18.8 ตัวอย่าง
ระบบเดียวกันจัด package 4 แบบ:
- Layer: `presentation/`, `domain/`, `data/`
- Feature: `order/`, `catalog/`, `payment/`
- Component: `order-component/`, `catalog-component/`
- Domain: `sales/`, `inventory/`, `shipping/`

แต่ละแบบมีผลต่อการ maintain และการทดสอบต่างกัน

18.9 การประยุกต์ใช้
- ใช้ workshop เลือกโครงสร้าง package
- ใช้ทบทวนโครงสร้างเดิม
- ใช้ใน architecture review

18.10 สรุป/ผลลัพธ์
ผู้อ่านเลือกแนวทาง package ได้อย่างมีเหตุผล และเข้าใจผลกระทบ

18.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 19
- ไปสู่เล่ม 3–4

บทที่ 19: Aligning Software Architecture and Code

19.1 เรื่อง
การทำให้สถาปัตยกรรมและโค้ดสอดคล้องกันอย่างต่อเนื่อง

19.2 วัตถุประสงค์
- วางกลไกกำกับ alignment
- ทำให้ alignment ไม่ใช่เรื่องครั้งเดียว
- เตรียมพื้นฐานสำหรับการดูแลระยะยาว

19.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- วางกลไกตรวจสอบ alignment
- ใช้เครื่องมือช่วยตรวจ
- รักษา alignment อย่างต่อเนื่อง

19.4 หัวข้อหลัก

19.4.1 Architecture Review
- ทบทวนสถาปัตยกรรมเป็นระยะ
- เทียบกับ drivers และ decision record
- ระบุ drift และวางแผนแก้

19.4.2 Fitness Function เบื้องต้น
- การทดสอบอัตโนมัติที่ตรวจคุณสมบัติสถาปัตยกรรม
- เช่น ตรวจว่าไม่มี cyclic dependency
- ตรวจว่า layer ไม่ข้ามกัน

19.4.3 Static Analysis
- ใช้เครื่องมือตรวจ dependency
- ตรวจ package structure
- ตรวจ coding style

19.4.4 Refactoring เพื่อรักษา Alignment
- refactor เมื่อพบ drift
- ทำอย่างต่อเนื่อง ไม่ปล่อยสะสม
- ใช้ test เป็นตาข่ายรองรับ

19.5 ข้อดีของบท
- ทำให้ alignment ไม่ใช่เรื่องครั้งเดียว
- ลด drift สะสม
- เตรียมพร้อมสำหรับการเปลี่ยนแปลง

19.6 ข้อเสีย/ข้อจำกัด
- ต้องมีเครื่องมือและวินัยทีม
- อาจต้องลงทุนเวลาในช่วงเริ่มต้น

19.7 ข้อควรระวัง
- ปล่อย drift สะสมจนแก้ยาก
- ใช้เครื่องมือโดยไม่เข้าใจ
- ละเลยการ review ด้วยมนุษย์

19.8 ตัวอย่าง
ใช้ ArchUnit (Java) ตรวจว่า:
- `domain` ไม่ import `infrastructure`
- ไม่มี cyclic dependency ระหว่าง package
- `api` ไม่เรียก `repository` โดยตรง

19.9 การประยุกต์ใช้
- ใช้ตั้งกลไกตรวจสอบใน CI
- ใช้ใน architecture review
- ใช้เป็น guardrail ของทีม

19.10 สรุป/ผลลัพธ์
ผู้อ่านวางกลไกกำกับ alignment ได้ และรักษา alignment อย่างต่อเนื่อง

19.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 20
- ไปสู่เล่ม 7 (การบริหารโครงการที่รวม quality)

ส่วนที่ 5: การ dokument และกรณีศึกษา

บทที่ 20: Documenting Software Architecture

20.1 เรื่อง
การ dokument สถาปัตยกรรมอย่างพอดี — document อะไร สำหรับใคร

20.2 วัตถุประสงค์
- ให้ผู้อ่าน dokument อย่างมีวัตถุประสงค์
- ลดเอกสารเกินจำเป็น
- ทำให้เอกสารใช้งานได้จริง

20.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- จัดทำเอกสารสถาปัตยกรรมขั้นต่ำได้
- เลือก document ให้เหมาะกับผู้ชม
- ใช้ ADR บันทึกการตัดสินใจ

20.4 หัวข้อหลัก

20.4.1 Document อะไร
- Context diagram
- Component diagram
- Deployment diagram
- Decision record (ADR)
- Quality attributes และ drivers

20.4.2 สำหรับใคร
- Developer — ต้องการ component/interface
- PM — ต้องการภาพรวมและความเสี่ยง
- Operations — ต้องการ deployment
- Business — ต้องการ context

20.4.3 Context Diagram, Component Diagram, Deployment Diagram
- **Context** — ระบบและระบบภายนอก

- **Component** — องค์ประกอบภายใน
- **Deployment** — การติดตั้งบน infrastructure

20.4.4 Architecture Decision Record (ADR) เบื้องต้น
- บันทึกการตัดสินใจสำคัญ
- ระบุ context, decision, consequences
- เก็บใน repository คู่กับโค้ด

20.5 ข้อดีของบท
- ลดเอกสารเกินจำเป็น
- เพิ่มการสื่อสาร
- ทำให้การตัดสินใจตรวจสอบย้อนกลับได้

20.6 ข้อเสีย/ข้อจำกัด
- ต้องปรับตามผู้ชม
- ต้องอัปเดตอย่างต่อเนื่อง

20.7 ข้อควรระวัง
- เอกสารมากเกินไปแต่ไม่ถูกใช้
- เอกสารไม่ตรงกับโค้ด
- ละเลย ADR

20.8 ตัวอย่าง
ชุดเอกสารสถาปัตยกรรมขนาดเล็ก:
- Context diagram 1 หน้า
- Component diagram 1 หน้า
- ADR 3 ฉบับสำหรับการตัดสินใจสำคัญ
- Quality attributes list 1 หน้า

20.9 การประยุกต์ใช้
- ใช้ตั้ง template เอกสารของทีม
- ใช้ใน architecture review
- ใช้เป็น onboarding material

20.10 สรุป/ผลลัพธ์
ผู้อ่านจัดทำเอกสารสถาปัตยกรรมขั้นต่ำได้ และเลือก document ให้เหมาะกับผู้ชม

20.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 21–22
- ไปสู่เล่ม 7 (การสื่อสารในโครงการ)

บทที่ 21: Simple Sketches for Diagramming your Software Architecture

21.1 เรื่อง
การวาด diagram อย่างง่ายแต่สื่อสารได้

21.2 วัตถุประสงค์
- ให้ผู้อ่านวาด diagram เพื่อสื่อสารสถาปัตยกรรม
- ลด diagram ที่รกและไม่สื่อความหมาย
- เตรียมพื้นฐานสำหรับ context/component diagram

21.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- วาด context/component diagram เบื้องต้นได้
- ใช้สัญลักษณ์พื้นฐานอย่างสม่ำเสมอ
- หลีกเลี่ยง diagram ที่รก

21.4 หัวข้อหลัก

21.4.1 สัญลักษณ์พื้นฐาน
- กล่อง = องค์ประกอบ
- ลูกศร = dependency/การสื่อสาร
- ขอบเขต = boundary
- สี = ประเภท (ถ้าจำเป็น)

21.4.2 Context → Container → Component
- **Context** — ระบบและระบบภายนอก
- **Container** — แอป/บริการ/ฐานข้อมูล
- **Component** — องค์ประกอบภายใน container

21.4.3 การแสดง Dependency
- ลูกศรชี้ทิศทาง dependency
- ระบุประเภท (sync/async)
- หลีกเลี่ยงลูกศรไขว้กันมาก

21.4.4 การหลีกเลี่ยง Diagram ที่รก
- แสดงเฉพาะสิ่งที่จำเป็น
- ใช้ legend
- แยก diagram ตามระดับ

21.5 ข้อดีของบท
- ใช้งานได้ทันทีในทีม
- ลดเวลาในการทำความเข้าใจ
- ช่วยในการสื่อสารกับผู้มีส่วนได้ส่วนเสีย

21.6 ข้อเสีย/ข้อจำกัด
- ไม่ได้ครอบคลุม notation ทั้งหมด
- ต้องฝึกฝนบ้าง

21.7 ข้อควรระวัง
- diagram ไม่มี legend หรือขอบเขต
- แสดงรายละเอียดมากเกินไป

- ไม่อัปเดตเมื่อสถาปัตยกรรมเปลี่ยน

21.8 ตัวอย่าง
Simple sketch ของระบบ e-commerce:
- กล่องกลาง: "E-commerce System"
- กล่องรอบ: "Customer", "Payment Gateway", "Shipping Provider"
- ลูกศร: การสื่อสาร
- Legend: อธิบายสัญลักษณ์

21.9 การประยุกต์ใช้
- ใช้ใน workshop และเอกสาร
- ใช้ใน architecture review
- ใช้เป็น onboarding material

21.10 สรุป/ผลลัพธ์
ผู้อ่านวาด diagram เพื่อสื่อสารได้ และหลีกเลี่ยง diagram ที่รก

21.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 22
- ไปสู่เล่ม 2–4 (diagram ของแต่ละแนวทาง)

บทที่ 22: Case Study: techtribes.je – Context

22.1 เรื่อง
กรณีศึกษาการทำความเข้าใจ context ของระบบจริง

22.2 วัตถุประสงค์
- ประยุกต์แนวคิด context และ drivers กับกรณีศึกษา
- ทำให้เห็นการวิเคราะห์ระบบจริง
- เตรียมผู้อ่านสำหรับการวิเคราะห์ระบบของตนเอง

22.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- เขียน context diagram ได้
- ระบุ drivers และ constraints ได้
- วิเคราะห์บทเรียนจากกรณีศึกษา

22.4 หัวข้อหลัก

22.4.1 System Context
techtribes.je เป็นระบบที่รวบรวมข้อมูลชุมชนเทคโนโลยีบนเกาะเจอร์ซีย์ ประกอบด้วย:
- เว็บไซต์
- บล็อก
- ทวิตเตอร์
- กิจกรรม

22.4.2 ผู้ใช้และระบบภายนอก
- ผู้ใช้ทั่วไป – ดูข้อมูล
- สมาชิกชุมชน – มีโปรไฟล์
- ระบบภายนอก – Twitter API, บล็อกต่างๆ

22.4.3 Constraints
- งบประมาณจำกัด
- ทีมเล็ก
- ข้อมูลจากภายนอกเปลี่ยนแปลงได้

22.4.4 Architectural Drivers
- ความง่ายในการ maintain
- ความสามารถในการรวบรวมข้อมูลจากหลายแหล่ง
- ต้นทุนต่ำ

22.4.5 บทเรียนจากกรณีศึกษา
- Context diagram ช่วยให้เห็นขอบเขตชัด
- Drivers ที่ชัดช่วยเลือกสถาปัตยกรรม
- ระบบเล็กก็ต้องการสถาปัตยกรรมที่เหมาะสม

22.5 ข้อดีของบท
- เชื่อมแนวคิดทั้งหมดกับตัวอย่างจริง
- ทำให้เห็นการประยุกต์ใช้ในบริบทจริง
- เป็นแบบฝึกหัดสำหรับผู้อ่าน

22.6 ข้อเสีย/ข้อจำกัด
- เป็นกรณีศึกษาเดียว
- บางรายละเอียดอาจไม่ครบ

22.7 ข้อควรระวัง
- มองข้ามระบบภายนอกและผู้ใช้งาน
- ละเลย constraints
- คัดลอกโดยไม่ปรับบริบท

22.8 ตัวอย่าง
Context diagram ของ techtribes.je:
- กล่องกลาง: "techtribes.je"
- กล่องรอบ: "Visitor", "Member", "Twitter API", "Blogs", "News Feeds"

22.9 การประยุกต์ใช้
- ใช้เป็นแบบฝึกหัดวิเคราะห์ระบบ
- ใช้เป็น template สำหรับระบบของตนเอง
- ใช้ใน workshop

22.10 สรุป/ผลลัพธ์
ผู้อ่านวิเคราะห์ context ของระบบได้ และเห็นบทเรียนจากการประยุกต์ใช้

22.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่บทที่ 23
- ไปสู่เล่ม 2–7 (การใช้ context ในแต่ละด้าน)

บทที่ 23: การประยุกต์ใช้และสรุปเล่ม

23.1 เรื่อง
การนำเนื้อหาทั้งเล่มไปใช้ในองค์กร

23.2 วัตถุประสงค์
- สรุปสาระสำคัญของเล่ม
- แปลงความรู้เป็นแผนปฏิบัติ
- เชื่อมไปยังเล่มถัดไป

23.3 เป้าหมาย
เมื่อจบบท ผู้อ่านสามารถ:
- มีแผนประยุกต์ใช้ในองค์กร
- จัด workshop ตามแม่แบบในเล่ม
- รู้ว่าต้องไปอ่านเล่มใดต่อ

23.4 หัวข้อหลัก

23.4.1 Workshop: Architecture Context Canvas
- ใช้ภาคผนวก A
- ระบุ context, ผู้ใช้, ระบบภายนอก, constraints, drivers
- ใช้เป็นจุดเริ่มต้นของ architecture review

23.4.2 Workshop: Style/Pattern Decision
- ใช้ภาคผนวก B
- เปรียบเทียบทางเลือก
- บันทึกการตัดสินใจ

23.4.3 Workshop: Package Alignment Review
- ใช้ภาคผนวก D
- ตรวจ package structure กับสถาปัตยกรรม
- ระบุ drift และวางแผนแก้

23.4.4 สรุปสาระสำคัญ
- สถาปัตยกรรมคือโครงสร้าง+หลักการ+เหตุผล
- การตัดสินใจต้องอิง drivers และ trade-offs
- สถาปัตยกรรมต้องสะท้อนในโค้ด
- Alignment ต้องดูแลอย่างต่อเนื่อง
- Dokument อย่างพอดี

23.4.5 เชื่อมไปเล่ม 2–4
- เล่ม 2: Microservices
- เล่ม 3: DDD
- เล่ม 4: Clean Architecture
- เล่ม 5: ความเสี่ยง
- เล่ม 6: POC
- เล่ม 7: การบริหารโครงการ

23.5 ข้อดีของบท
- ปิดเล่มด้วยการลงมือทำ
- ทำให้ความรู้แปลงเป็นแผนปฏิบัติ
- เชื่อมโยงกับเล่มถัดไปอย่างชัดเจน

23.6 ข้อเสีย/ข้อจำกัด
- ต้องปรับตามบริบทองค์กร
- ต้องมีผู้นำ workshop

23.7 ข้อควรระวัง
- อย่านำไปใช้แบบคัดลอกโดยไม่ปรับบริบท
- อย่าทำ workshop ครั้งเดียวแล้วจบ
- อย่าละเลยการติดตามผล

23.8 ตัวอย่าง
แผนนำไปใช้ใน 30 วัน:
- สัปดาห์ 1: Architecture Context Canvas
- สัปดาห์ 2: Style/Pattern Decision
- สัปดาห์ 3: Package Alignment Review
- สัปดาห์ 4: วางแผนปรับปรุงและติดตามผล

23.9 การประยุกต์ใช้
- ใช้เป็นแผนดำเนินงานหลังอ่านจบ
- ใช้เป็น checklist ของทีม
- ใช้เป็นพื้นฐานสำหรับเล่มถัดไป

23.10 สรุป/ผลลัพธ์
ผู้อ่านมีแผนประยุกต์ใช้ในองค์กร และรู้เส้นทางการอ่านต่อ

23.11 เชื่อมโยงกับเล่มอื่น
- ไปสู่เล่ม 2–7 ทั้งหมด

ภาคผนวก

ภาคผนวก A: แม่แบบ Architecture Context Canvas

| หัวข้อ | คำอธิบาย | ตัวอย่าง |

```
|---|---|---|
| ระบบคืออะไร | วัตถุประสงค์และขอบเขต | ระบบ e-commerce สำหรับ SME |
| ผู้ใช้/actor | ใครใช้งานหรือเกี่ยวข้อง | ลูกค้า, ผู้ดูแลร้าน, ผู้ขนส่ง |
| ระบบภายนอก | ระบบที่เชื่อมต่อ | Payment Gateway, Shipping API |
| Constraints | ข้อจำกัดทางธุรกิจ/เทคนิค/กฎหมาย | งบประมาณ, PDPA, ทีม 5 คน |
| Quality attributes | คุณภาพที่สำคัญ | Security, Availability, Maintainability |
| Architectural drivers | ปัจจัยขับเคลื่อนการตัดสินใจ | Time-to-market, Security |
| ความเสี่ยงเบื้องต้น | ความเสี่ยงที่มองเห็น | การพึ่งพา Payment Gateway เดียว |

---

## ภาคผนวก B: แม่แบบ Style/Pattern Decision Record

| หัวข้อ | รายละเอียด |
|---|---|
| ชื่อ decision | เลือกสถาปัตยกรรมสำหรับระบบ e-commerce |
| ปัญหา | ต้องเลือกระหว่าง monolith และ microservices |
| ทางเลือก | 1. Modular monolith 2. Microservices 3. Component-based monolith |
| เหตุผล | อ้างอิง drivers: time-to-market สูง, ทีมเล็ก |
| ข้อดี/ข้อเสีย | Modular monolith: เริ่มง่าย, ยัง modular ได้ |
| ความเสี่ยง | อาจต้องแยกบริการภายหลัง |
| การตัดสินใจ | เลือk Modular monolith |
| ผู้รับผิดชอบ | Architect + Tech Lead |
| วันที่ | 2026-10-30 |

---

## ภาคผนวก C: แม่แบบ Component & Dependency Map

| Component | ความรับผิดชอบ | Interface/Contract | Dependency | Owner |
|---|---|---|---|---|
| Catalog | จัดการสินค้า | CatalogService | - | ทีม A |
| Order | จัดการคำสั่งซื้อ | OrderService | Catalog, Payment | ทีม B |
| Payment | จัดการชำระเงิน | PaymentService | - | ทีม C |
| Notification | แจ้งเตือน | NotificationService | - | ทีม A |

---

## ภาคผนวก D: Package Alignment Checklist

- [ ] Package สื่อ component/domain หรือไม่
- [ ] Dependency direction ถูกต้องหรือไม่
- [ ] มี cyclic dependency หรือไม่
- [ ] Test ตรง boundary หรือไม่
- [ ] ชื่อ package สอดคล้องกับสถาปัตยกรรมหรือไม่
- [ ] มี package ที่ใหญ่หรือเล็กเกินไปหรือไม่
- [ ] มีการเรียกข้าม layer โดยตรงหรือไม่
```

```
- [ ] เอกสารกับโค้ดตรงกันหรือไม่

---

## ภาคผนวก E: Diagram Checklist

- [ ] มีชื่อ diagram
- [ ] มี legend
- [ ] มีขอบเขต (boundary)
- [ ] มีทิศทาง dependency
- [ ] มีคำอธิบายประกอบ
- [ ] ไม่รกเกินไป
- [ ] ตรงกับโค้ดหรือเอกสารล่าสุด
- [ ] ระบุระดับ (context/container/component)

---

## ภาคผนวก F: Glossary กลาง

| คำ | ความหมายโดยย่อ |
|---|---|
| Software Architecture | โครงสร้างหลักและหลักการของระบบ |
| Architectural Style | แนวทางจัดโครงสร้างระดับสูง |
| Architectural Pattern | รูปแบบการแก้ปัญหาเชิงสถาปัตยกรรม |
| Design Pattern | รูปแบบการแก้ปัญหาเชิงออกแบบโค้ด |
| Component | หน่วยสถาปัตยกรรมที่มี boundary |
| Module/Package | หน่วยจัดกลุ่มโค้ด |
| Layer | ชั้นความรับผิดชอบ |
| Boundary | ขอบเขตความรับผิดชอบ |
| Contract | ข้อตกลงการสื่อสารระหว่าง component |
| Quality Attribute | คุณภาพของระบบ เช่น scalability, maintainability |
| Architectural Driver | ปัจจัยขับเคลื่อนการตัดสินใจ |
| ADR | Architecture Decision Record |
| Alignment | ความสอดคล้องระหว่างสถาปัตยกรรมกับโค้ด |
| Drift | ภาวะที่เอกสารกับโค้ดไม่ตรงกัน |
| Fitness Function | การทดสอบอัตโนมัติที่ตรวจคุณสมบัติสถาปัตยกรรม |
| Coupling | ระดับการพึ่งพากันระหว่าง component |
| Cohesion | ระดับความเกี่ยวข้องกันภายใน component |
| Modular Monolith | Monolith ที่แบ่ง module ชัดเจน |
| Bounded Context | ขอบเขตของ domain ที่มีภาษาและโมเดลเฉพาะ |
| Ubiquitous Language | ภาษากลางที่ทีมและผู้เชี่ยวชาญธุรกิจใช้ร่วมกัน |

---

# เกณฑ์ประเมินความสำเร็จของเล่ม

| เกณฑ์ | ตัวชี้วัด | น้ำหนัก |
```

```
|---|---|---|
| ความเข้าใจ | อธิบาย style/pattern/design pattern ต่างกันได้ | 20% |
| การนำไปใช้ | มี diagram และ package map ของระบบตนเอง | 25% |
| การตัดสินใจ | มี decision record อย่างน้อย 1 ฉบับ | 20% |
| ความสอดคล้อง | ตรวจ alignment ระหว่างสถาปัตยกรรมกับโค้ดได้ | 20% |
| การส่งต่อ | เข้าใจว่าบทใดเชื่อมไปเล่ม 2, 3, 4 | 15% |

---

# สรุปเล่ม 1

เล่ม 1 ทำหน้าที่เป็นฐานของทั้งชุด ผู้อ่านจะได้:

1. **ภาษากลาง** — คำนิยามและความแตกต่างของ style/pattern/design pattern
2. **ภาพรวมรูปแบบสถาปัตยกรรม** — monolith, component-based, layered, microservices, clean architecture
3. **วิธี dokument** — context diagram, component diagram, ADR
4. **วิธีทำให้สถาปัตยกรรมสะท้อนในโค้ด** — mapping, coding style, package alignment
5. **เครื่องมือตรวจสอบ** — checklist, fitness function, architecture review
6. **กรณีศึกษา** — techtribes.je เป็นตัวอย่างการประยุกต์ใช้

เมื่ออ่านจบ ผู้อ่านควรมีพื้นฐานที่แข็งพอสำหรับการตัดสินใจในเล่ม 2–7 และมีเครื่องมือที่นำไปใช้ได้ทันทีในองค์กร

---
```